# COVENTRY UNIVERSITY

Faculty of Engineering, Environment and Computing

School of Computing, Electronics and Mathematics

**MSC. CYBER SCECURITY (EECT117)**

**CYBER SECURITY INDIVIDUAL PROJECT - 7030CEM**

# A COMPARATIVE STUDY OF MACHINE LEARNING METHODS FOR URL BASED PHISHING DETECTION


***by***

**AUTHOR: LIJI SAMUEL**

**STUDENT ID:13163231**

**EMAIL ID : samuell@coventry.ac.uk**

**SUPERVISOR: CHRISTO PANCHEV**


Submitted in partial fulfilment of the requirements for the Degree of Master of Science in Cyber Security.

Academic Year : 2022/2023

# Declaration of Originality

I declare that this project is all my own work and has not been copied in part or in whole from any other source except where duly acknowledged. As such, all use of previously published work (from books, journals, magazines, internet etc.) has been acknowledged by citation within the main report to an item in the References or Bibliography lists. I also agree that an electronic copy of this project may be stored and used for the purposes of plagiarism prevention and detection.

# Statement of copyright

I acknowledge that the copyright of this project report, and any product developed as part of the project, belong to Coventry University. Support, including funding, is available to commercialise products and services developed by staff and students. Any revenue that is generated is split with the inventor/s of the product or service. For further information please see www.coventry.ac.uk/ipr or contact ipr@coventry.ac.uk.

# Statement of ethical engagement

I declare that a proposal for this project has been submitted to the Coventry University ethics monitoring website (https://ethics.coventry.ac.uk/) and that the application number is listed below (Note: Projects without an ethical application number will be rejected for marking)

Signed: Liji Samuel  Date: 07/08/2023

Please complete all fields.

| | |
|---|---|
| First Name: | Liji |
| Last Name: | Samuel |
| Student ID number | 13163231 |
| Ethics Application Number | P158822 |
| 1 st Supervisor Name | Christo Panchev |
| 2 nd Supervisor Name | Derrick Newton |

**This form must be completed, scanned and included with your project submission to Turnitin. Failure to append these declarations may result in your project being rejected for marking**

## ABSTRACT

Website phishing poses a serious risk to cyber security because imposters pretend to be reputable websites in order to steal information. Phishing and legal websites can be identified from URLs using machine learning (ML) techniques like Random Forest, Decision Tree, SVM, K Nearest neighbor, Logistic Regression and XGBoost. Data of both legitimate and phishing URLs will be collected from the internet and websites like www.phishtank.com instead of using already available datasets in which the URLs will be expired. The raw URLs gathered from the internet will be processed to extract features including HTML-based features, domain-related characteristics, and address bar-related features. Data from trustworthy and phishing URLs will be gathered, cleaned, and trained using ML models. These models will be examined utilising metrics and performance evaluation measures like accuracy, precision, recall, F1_score. To safeguard the security of the internet and the entire world, the outputs and results will be tallied and compared.

# Table of Contents

**Abstract** 1
**Table of Contents** 2
**List of Figures** 5
**List of Tables** 7
**Acknowledgements** 8
**1 Introduction** 9
**1.1 Background to the Project** 9
**1.2 Phishing Attack Statistics** 11
**1.3 Problem Statement** 11
**1.4 Research Question** 13
**1.5 Aim** 13
**1.6 Relevance** 14
**1.7 Overview of the Project** 15
**2 Literature Review** 16
**3 Methodology** 21
**3.1 Data Collection** 22
**3.2 Data Cleaning and Preprocessing** 22
**3.3 Feature Extraction** 22
**3.4 Train Machine Learning Models** 25
**4 Requirements** 32
**4.1 System Requirements** 32
**4.2 Data Collection** 33
**5 Analysis** 34
**6 Design** 43
**7 Implementation** 44
**8 Testing** 59
**9 Project Management** 61
**9.1 Project Schedule** 61
**9.2 Risk Management** 61
**9.3 Quality Management** 61
**9.4 Social, Legal, Ethical and Professional Considerations** 62
**10 Critical Appraisal** 63

**11 Conclusions 63**

**11.1 Achievements 65**

**11.2 Future Work 65**

**12 Student Reflections 67**

**References 68**

**Appendix A – Project Specification ..... 1**

**Appendix B – Interim Progress Report and Meeting Records..... 1**

**Appendix C – Project Presentation..... 1**

**Appendix D – Certificate of Ethics Approval..... 1**

# List of Figures

**Figure 1.1: Original Vs Phishing Facebook Login Page 10**
**Figure 3.1: Data Flow Diagram 21**
**Figure 3.2: KNN explanation 26**
**Figure 3.3: Decision Tree 27**
**Figure 3.4: Random forest architecture 28**
**Figure 3.5: Random forest classifier procedures 28**
**Figure 3.6 : SVM 29**
**Figure 3.7: Logistic regression 30**
**Figure 5.1: Confusion matrix 34**
**Figure 5.2 Performance Metrics of K-NN 35**
**Figure 5.3 Performance metrics of Decision Tree 36**
**Figure 5.4 Performance metrics of Random Forest 38**
**Figure 5.5 Performance metrics of SVM 39**
**Figure 5.6 Performance metrics of Logistic Regression 40**
**Figure 5.7 Performance metrics of XGBoost 41**
**Figure 6.1 Data Flow Diagram of the System Design 43**
**Figure 6.2: Training and Testing 43**
**Figure 7.1 : The phishtank website where the phishing URLs are collected 44**
**Figure 7.2: Phishing URLs collected 44**
**Figure 7.3: Legitimate URLs collected 45**
**Figure 7.4: The final dataset 46**
**Figure 7.5: Final dataset with 30 features. 46**
**Figure7.6 Final Dataset 48**
**Figure 7.7 Heatmap Visualization 49**
**Figure 7.8 Splitting dataset in Train – Test set ratio 50**
**Figure 7.9 Screenshot of Training K-NN 51**
**Figure 7.10 Screenshot of Prediction 51**
**Figure 7.11 Screenshot of Training Vs Test Accuracy of KNN 52**
**Figure 7.12 Plot of Training vs Test accuracy 52**
**Figure 7.13 Screenshot of Training Vs Test Accuracy of Decision Tree 53**
**Figure 7.14 Plot of training Vs Test Accuracy of Decision tree 54**

**Figure 7.15 Screenshot of Training Vs Test Accuracy of Random Forest 54**

**Figure 7.16 Plot of Training Vs Test Accuracy of Random Forest 55**

**Figure 7.17 Screenshot of Training Vs Test Accuracy of SVM 56**

**Figure 7.18 Screenshot of Training Vs Test Accuracy of Logistic Regression 56**

**Figure 7.19 Screenshot of Training Vs Test Accuracy of XGBoost 57**

**Figure 8.1: Final Testing 59**

**Figure 9.1 Gantt Chart 61**

# List of Tables

**Table 5.1: Comparison of ML Methods 42**

# ACKNOWLEDGMENTS

I would want to express my heartfelt gratitude and appreciation to everyone who helped me finish this study on "A Comparative Study of Machine Learning Methods for URL-Based Phishing Detection." This project would not have been possible without the assistance of numerous people and resources.

I would like to start by expressing my sincere gratitude to my respected supervisor Christo Panchev, whose vital advice, mentorship, and support throughout the research process have been crucial in determining the focus and calibre of my work. I would also want to thank Derrick Newton, who has been a consistent source of assistance and support throughout the project.

I would like to express my gratitude to Coventry University for granting me access to the resources that were essential for carrying out the experiments and doing the analyses.

I also thank the individuals who voluntarily gave their time and data to compile the dataset that was used in this study.

Last but not least, I would like to express my gratitude to my family for their unfailing support, patience, and inspiration during this academic journey.

Finally, I would like to express my gratitude to everyone who has contributed in some way to this research. Their efforts have unquestionably made a positive impact on the outcome of this endeavour.

Thank you,

Sincerely,
Liji Samuel

# 1 INTRODUCTION

With the creation of online communication and the worldwide web, one of the most serious dangers to current cyber security has been website phishing. It is a cyber fraud in which an imposter will be faking a legitimate website in its content such as the website of a bank or any other organization. The fake one will have the complete features of the original website including colour theme, logo, texts, URL, and appearance so distinguishing the fake one and legitimate one will be challenging (Sang et al., 2019).

Identity theft is a crime in which the offender sends a fake email or website URL that looks to be from a reliable source or respected organisation and requests personal information such as bank ownership, usernames, numbers, addresses, capital card details, and more. When a user of the internet is prompted to provide personal information, fraudulent emails and websites can appear to be quite authentic. E-mail, SMS, quick messengers, social media platforms, VoIP, and other methods are used to spread phishing scams. However, accessing the e-mail link allows you to learn more about these attacks and phishing scams. Additionally, identity theft attacks are altering considerably in the modern world.

## 1.1 BACKGROUND TO THE PROJECT

Theft of sensitive information remains an important risk to security, and a substantial percentage of online users become victims to this fraud. Furthermore, such attacks damage not only internet users but also firms that provide online financial services. This is due to the fact that online service providers frequently suffer reputational and financial harm when users become victims of identity theft.

Every year, phishing costs consumers of internet billions of dollars. It refers to the methods employed by criminals who steal identities to sift through an array of unknowing Internet users for sensitive data. Phishers gain access credentials and password for banking and personal accounts by forging emails and using phishing software. To deceive consumers into entering fake websites and giving away sensitive financial data such as login credentials and passwords, social engineering strategies include counterfeit emails that appear to be from trusted organisations and groups. Technically deceptive tactics may include systems that capture clients login information and passwords for network accounts in order to obtain credential directly by putting malicious software onto machines.




***Figure 1.1 Original Vs Phishing Facebook Login Page***

The homepage of the well-known website www.facebook.com and its phish are shown in Figure 1.1. The second graphic shows a website that resembles Facebook but is actually a website that promotes phishing scams. A user can enter his personal information thinking the second website is the real Facebook.com. Thus, the Phisher has the ability to steal that information and utilise it maliciously (James et al, 2013).

A cybercrime website hits online businesses, banks, Web users, and governments, so it has become a national security issue. It is necessary that this attack be detected early. However, it is difficult to see these attacks because of the new methods used by criminals to steal sensitive information to commit crimes. For successful criminal detection of identity theft to be achieved, it must be obtained with the highest accuracy and in the shortest possible time. The most common method of detecting identity theft involves black listing and white listing.

Criminals use URLs for stealing sensitive information can be obtained with the concept of machine learning, which can be used continuously to prevent such attacks. First, machines used to follow instructions given to man, but now people can train a machine to learn from previous data, build a prediction model and perform very fast, and this is known as machine learning. It is basically the use of tools and technologies that can be used effectively and efficiently. Machine learning is used to make one's work easier, faster, and more accessible by learning from past data and working efficiently now. Machine language uses data to answer questions. This data usage is called the Training process, which uses our data to create and transform it into a predictable model. We may provide data on our machine, and it will assist in identifying and locating sensitive web theft websites.

## 1.2 PHISHING ATTACKS STATISTICS

Hacking is one of the most prevalent types of internet identity theft fraud, producing both immediate and lasting economic damage. In 2012, there were approximately 33,000 phishing attacks per month worldwide, leading to an entire expense of $687 billions of dollars (Sandi, 2019). A phishing scenario occurred in June 2004. Customers were notified by the Royal Bank of Canada that they had received bogus emails purporting to be from the Royal Bank, asking customers to verify account numbers and personal identification codes (PINs) via a hyperlink included in the email. The fraudulent email stated that the recipient would lose entry to his account if he hadn't clicked on the link provided and submit his client cardholder credentials and PIN. These emails were sent out within a week of a computer malfunction that prevented client accounts from being updated (AbuTaha et al, 2019).The United States of America remained the most popular country host fraudulent websites in the fourth quarter of 2012. This is primarily because the USA hosts a large proportion of the the world's web pages and domain names. Financial services remain an extremely frequently targeted industry sector by phishers.

This project investigates and develops methods to detect phishing URLs using Machine Learning techniques. A number of machine learning models are trained to detect phishing URLs from unknown URLs. The phishing URL detection algorithms were developed on well-prepared data obtained through web scraping. he phishing and genuine URLs were gathered from a variety of sources, including www.phishtank.com and www.kaggle.com. This URL list is passed through a well-engineered preprocessing stage where around 30 features are extracted and preprocessed. The models are trained using this efficient dataset. The method of feature extraction and preprocessing processes are critical. Machine Learning models such as KNN (K-Nearest Neighbour), Support Vector Machine(SVM), Logistic Regression, Decision Tree, Random Forest, and XGBoost are trained. They are assessed using criteria including as accuracy, precision, recall, and F1-score. The trained models for phishing detection are saved as pickle files and are tested.

## 1.3 PROBLEM STATEMENT

Cybersecurity experts continue to find phishing assaults to be a severe problem that affects people and businesses all around the world. To force users into disclosing personal information like usernames, passwords, and financial information, these assaults employ deceptive tactics. Phishing attack detection and mitigation are crucial for the security of online users and the integrity of sensitive data. Despite the promise demonstrated by machine learning algorithms for recognising

phish URLs, there continue to be issues and limits in the available literature that must be addressed in order to further enhance the effectiveness of URL-based detection of phishing.

Cybercriminals continue to utilise fake URLs to coerce users into disclosing sensitive information or carrying out destructive deeds, posing a severe threat to people and organisations through phishing assaults. These advanced phishing efforts are difficult to identify and prevent using conventional security methods. In order to address this issue, the primary goal is to conduct an in-depth comparison of machine learning approaches for URL-based phishing detection. The study's main purpose is to evaluate how well-known algorithms for machine learning, such as K nearest neighbours (KNN), Support Vector Machine (SVM), Decision Tree, Random Forest, Logistic Regression, and XGBoost, perform in properly identifying phishing URLs and distinguishing them from legitimate URLs.

In order to accomplish this goal, a dataset of URLs that is both diverse and representative and includes both legitimate and phishing cases will be constructed . To guarantee its legitimacy and applicability, the dataset will be gathered from dependable repositories and real-world circumstances. In order to improve the algorithms capacity for making precise predictions, data preparation techniques will be used to extract useful information from the URLs.

With feature selection, several sets of attributes ranging from 4 to 86, a variety of approaches in the literature, and achieved performance in the range of 86%-98%. The inclusion of a significant number of characteristics may make it difficult to choose a URL, may make things more complicated, and may indicate a dependence on a third party. A quick and clever detection method is required because phishing sites have a short lifespan.

This study will draw on past research while also analysing the performance of machine learning algorithms using standard measures such as accuracy, precision, recall, and F1 score. Also helps to enhance URL-based phishing detection approaches and strengthen cybersecurity defence against phishing attempts by using the collective knowledge of the cybersecurity community. By applying these Machine Learning algorithms would help to answer mainly two fundamental questions.

1. How effective is the new strategy in comparison to the current strategies?
2. How does the new method with feature selection affect things?

## 1.4 RESEARCH QUESTION

Q1: Which machine learning approach—among Decision Tree, Logistic Regression, SVM, KNN, XGBoost and Random Forest—is most successful at detecting URL-based phishing?

Q2: How efficient and accurate in terms of detection is the new method with feature selection compared to the old method without feature selection?

Q3: What practical ways might the study's findings be put to use to strengthen cybersecurity defences against phishing attempts in real-world situations?

## 1.5 AIM

The purpose of this research is to conduct a thorough comparison of machine learning approaches for URL-based identification of phishing attacks. The main goal is to evaluate the effectiveness and efficiency of well-known algorithms such as K-Nearest Neighbours (KNN), Support Vector Machine (SVM), Decision Tree, Random Forest, Logistic Regression, and XGBoost in correctly recognising and distinguishing phishing URLs from real URLs. In order to find the most reliable and effective method of phishing detection, the research takes into account a wide variety of algorithms.

The study also attempts to advance the findings of earlier investigations on the detection of URL-based phishing. The project intends to improve techniques, parameter tuning, and techniques for feature selection to enhance accuracy and efficiency of the selected algorithms by applying insights obtained from past studies.

**User requirement:**

The user requirement for a good phishing detector demands high accuracy, low false alarm and low latency. Because the system should detect and raise alert of phishing at the very second a user gets into a malicious URL. So, a slow algorithm or model will not work here. Accuracy and false alarms are crucial as the system should not identify and take action on a legitimate website which will badly affect the user experience.

From a developer point of view the whole system should not be complex to implement. The project should be implementable and deployable with minimum technology stack support.

- Collect phishing and legitimate URL data using python web scraping tools and extract the URL features from the dataset.
- Clean, preprocess, prepare the dataset to train the ML models.

- Develop and train the model using prepared data.
- Run tests and analyse the performance of artificial intelligence models on various parameters.

## 1.6 RELEVANCE

As digital technologies have developed, connection has improved but we have also become more vulnerable to cybersecurity dangers, such as phishing attempts. These attacks trick users into revealing private information by using misleading URLs. To shield people and organisations against these malevolent schemes as they get more sophisticated, strong detection techniques are required.

This study's relevance comes from its focus on the urgent cybersecurity issues that phishing assaults raise. The study seeks to determine the best efficient method to detect and distinguish phishing URLs from real ones by conducting a thorough comparative analysis of machine learning algorithms for URL-based phishing detection. Insights from this study's findings can help cybersecurity professionals strengthen their defences against phishing assaults by helping them create reliable and effective phishing detection systems. The findings have direct relevance to current cybersecurity practises, enabling experts to protect people and organisations from potential data breaches and financial losses. Several aspects that makes the research more relevant are:

1) Growing Phishing Threat: Phishing attacks, which target people, companies, and organisations, remain one of the most common cybersecurity dangers. To protect against these dangers, improved detection systems are becoming more and more necessary as phishing attempts get more sophisticated.

2) Machine learning developments: Phishing detection is one area where machine learning algorithms have demonstrated promising results. This study examines the possibility of several machine learning techniques to enhance the precision and effectiveness of phishing detection by utilising recent developments in the area.

3) Standardised Dataset: By using a standardised dataset that contains both phishing and valid URLs, the research ensures fair comparisons and unbiased evaluations of the algorithms performance. This method improves the findings' dependability and relevance to real-world circumstances.

4) Feature Selection: The study explores the impact of selecting features on the effectiveness of predictive models. Proper feature selection is crucial for finding informative properties of phishing URLs, allowing algorithms to make accurate predictions.

5) Practical Application: The research's conclusions are directly applicable to real-world cybersecurity practises. The creation of accurate and reliable phishing detection systems can be aided by discovering the most successful algorithms and feature selection approaches.

## 1.7 OVERVIEW OF THE PROJECT

The report is structured as follows:

Chapter 1 introduces the system and contains the motivation and aim.

Chapter 2 goes through the literature survey with a detailed study.

Chapter 3 describes the Methodology, which includes the methods used in the research.

Chapter 4 provides the requirements that is needed for the research to carry out

Chapter 5 discusses the analysis and results.

Chapter 6 is about research design and architecture.

Chapter 7 includes implementation part and its discussions

Chapter 8 deals with the testing carried out in various stages

Chapter 9 involves project management, the risk involved and the schedule of work

Chapter 10 deals with the critical appraisal that was faced during the research

Chapter 11 concludes the report

Chapter 12 student reflections

# 2 LITERATURE REVIEW

This chapter discusses the previous and existing works on malicious URL detection with different approaches.

There are many research papers applying various techniques such as URL ranking, etc for detecting phishing websites. The lack of performance of all these techniques gave rise to ML and DL algorithms to be used in phishing detection.

To distinguish legitimate websites from phishing ones, phishing prevention machine learning techniques pull features from numerous sources, which include URLs, page content, and search engines. A balanced dataset was employed in this investigation, which included 11430 Url and 87 retrieved attributes. The random forest approach outperformed other proposed strategies, with 97.0% accuracy. (Uddin et. al,2022) Decision tree, Random Forest, KNeighbors, Gaussian Nave Byes, and XGBoost were the five machine learning approaches compared. 56 is derived from the arrangement and syntax of URLs. The material on the related pages is used to generate 24. Seven are obtained from queries to other services. The information set has a balance having precisely one half of the URLs proving phishing and the other half being legitimate.

(Nandi Tultul, Afroz, & Hossain, 2022) built that can collect characteristics from a website's URL on the fly and employed four classification methods: K-Nearest Neighbour, the decision tree, support vector machine, and random forest. For dataset_small.csv, experts used 24748 points of data as training information and 10,607 input points as testing data. Twenty-three features were selected based upon the heuristics to evaluate when a website appears to be phishing or not. The random forest algorithm using the provided features performs best, with a 95.67% accuracy rate. (Sandi et. al, 2019) Use a substantial URL sample with 22 distinct characteristics that have been simplified to a lower set through different feature reduction techniques. They trained models such as Random Forest, Gradient Boosting, Neural Network, and Support Vector Machine (SVM) classifiers, and discovered that SVM had the highest accuracy in identifying phishing. They finished by comparing URL-based phishing detection methods to other known methods. (Abutaha et. al, 2021) proved XGBoost giving best performance on a dataset with 48 features. The dataset they used had 1056937 URL samples that are labeled as phishing and legitimate. They generated insightful features by using feature engineering techniques and reduced them using future reduction techniques. (Megha Agarwal et al.,2023) Kaggle data was used, which included 86 characteristics and 11,430 overall URLs, fifty percent of which were phishing and the other half were valid. They tested some ML models as well as a very simple DL model, a Multi layered Perceptron (MLP), but discovered that XGBoost has a higher accuracy of roughly 96%. The

famous Naive Bayes algorithm has been used by (Alrefaai et. al, 2022) Using a dataset, they collected URL, HTML, and other ancillary information features of the fraudulent website extracted from the URLs.

(Y. Xu et al., 2022) go for a Naive Bayes algorithm and train using a dataset. To improve accuracy, use the URL, HTML, and other irrelevant features of the phishing website. (Ahmed J et.al, 2021) uses an adaptive random forest algorithm with an extensively preprocessed dataset. They have evaluated the trained model using the confusion matrix and show that the model has only 0.02% of False Negatives and 0.002% of False Positives. The paper also shows that adaptive random forest modelling outperforms traditional algorithms like SVM, decision tree, and K-NN. They extracted sophisticated data from the webfile using data collecting and preprocessing. They also extract characteristics that show user behaviour. They incorporated all of the characteristics and tested the classification models using a matrix of uncertainty and several kinds of metrics that demonstrate the robustness of the approach they developed.

(A. Selamat et. al, 2022) in their research goes through all the possible approaches in phishing detection like visual similarity-based, heuristics-based, white list-based, and black list-based. They accomplished this by conducting a thorough literature review on 81 selected studies. The paper also discusses all the possible ML algorithms in phishing detection from basic models such as logistic regression to advanced sequential Deep Learning architectures like LSTM, and GRU. They start with the concert of phishing and the application of Deep Learning in Cybersecurity. Then goes to taxonomies of phishing detection, deep learning techniques, and some issues within the deep learning systems. They conclude with an empirical analysis of evaluation on all the deep learning algorithms and also study their issues. According to their findings, the most typical faults and issues with advanced deep learning algorithms include manual parameter adjustment, extensive training times, and poor detection accuracy.

(Zainab Alshing et. al, 2023) Concentrate solely on advanced techniques for deep learning like CNN (Convolutional Neural Network), LSTM (Long Short-Term Memory), and CNN-LSTM, a hybrid of the two. The study indicates that the three models obtain 99.2%, 97.6%, and 96.8% efficiency accuracies for CNN, LSTM-CNN, and LSTM, respectively. The work of (Jibrilla Tanimu. et.al, 2022) in involves training conventional ML algorithms with a dataset of 30000 URLs in each legitimate and phishing class. They mainly concentrate on feature extraction and successfully extracted 30 types of features. They collected the data from the official phishtank website (www.phishtank.org). In feature extraction, they have used PCA (Principal Component Analysis), Recursive feature elimination, and feature importance method.

(Buket Geyik et. al, 2021) have collected a huge dataset of more than 126000 URL samples and extracted 15 features for training. In their dataset, they have 30.2% phishing samples and 69.8% legitimate samples. They trained four models, Decision Tree, Random Forest, Logistics Regression, and Naive Bayes and got around 90% accuracy for all of them. They have also tabulated the run time taken to generate the output for each model. (Masurkar et.al, 2021) has a couple of datasets containing 10000 and 5000 samples. Both the datasets have 23 features extracted. The datasets are trained on Random forest, decision tree, XGBooxt, Gradient Boost, and Naive Bayes algorithms and tabulated the comparative result. XGBoost shows the highest accuracy of 98% on both of the datasets. One drawback is that both the datasets used by them are unbalanced that will make the trained model biased.

(Sundara Pandiyan S et. al, 2022) has 17 feature URL dataset trained on Decision Tree, Random Forest, XGBoost, Support Vector Machine (SVM), Light GBM Classifier, Multi-Layer Perceptron (MLP), and Catboost classifier. Their tabular analysis shows that Light GBM attains higher accuracy in testing than the rest. (Bhavani P et. al, 2022) in compares advanced Deep Learning models of this decade such as CNN LSTM and CNN Bi-LSTM, with conventional Machine Learning models like Logistic regression and XGBoost algorithms in URL-based phishing detection. The dataset used has more than 50000 samples and a huge amount of features extracted. The number of features reaches 50 here. They split the data into train sets and test sets at 80:20 ratio. The research concludes that the conventional algorithms, Logistic regression (91.89%), and XGBoost classifier (92%) outperform the Deep Learning algorithms, CNN-LSTM (57.85), and CNN-Bi-LSTM (56.36%) by a huge margin. (Alnemari, S et. al, 2023) uses an old dataset from the UCI repository to train and compare conventional machine learning models with Artificial Neural Network. They have done a series of preprocessing such as Min-Max normalization on the dataset to make them better for Machine Learning training. They claim that the MinMax normalisation method of the proposed model reduces the information within a range of [0, 1], which improves the predictive learning input quality. The dataset has approximately 11000 cases with 30 characteristics. It was separated 80/20 into training and testing sets, with c5-fold cross-validation producing the most favourable results in the most current research. The framework for forecasting is then developed using the technique of machine learning, which employs a range of learning approaches. This is especially useful when making predictions because using multiple models guarantees the findings are not biased towards a single model. To prove that Random Forest beats even Artificial Neural Network, all four models are assessed using four metrics: precision, recall, precision, and f1-score.

(Arun Kulkarni et. al, 2019) uses a very small dataset of URLs in case of number of instances. They developed four models: the decision tree, the Nave Bayesian classifier, the support vector machine (SVM), and the neural network. The algorithms were tested on a collection of data of 1,353 real-world URLs, every single one of them could be classed as authentic or phishing. The trial findings demonstrate that the classifiers were successful in differentiating between legitimate and bogus websites more than 90% of the time.

(Catal C et. al,2022) A systematic survey of 43 research papers found that URL-related data, third-party website information, content-related data, and email were the most commonly used data sources. Deep learning approaches such as DNN, CNN, and LSTM were the most commonly utilised. 72% of research did not employ feature selection techniques, and most strongly used dataset was PhishTank. Although Keras and Tensorflow were favoured frameworks, they were referenced in 46% of papers. (Adebowale et al,2022) The smart phishing warning system (IPDS) is a hybrid model of classification that was trained on a dataset of 1 million URL samples. The model obtains 93.28% accuracy and a detection process that takes 25 seconds on average, addressing the issue of big data sets and improved prediction performance.

(Yang R et. al, 2021) has suggested a unique methodology that utilises convolutional neural networks (CNN) and random forest (RF) to detect phishing websites. In the suggested method, URLs are converted into fixed-size matrices using character embedding techniques. This is fed into the model as data. At several stages, the CNN model is used as a feature extractor. Finally, the vectors of features are utilised for learning the RF classifier. They used two datasets of 47210 and 83857 samples to assess the performance of the model employing accuracy, precision, recall, and f1-score, and the proposed model surpassed all traditional models. (Ping Yi et. al, 2020) conducted URL-based phishing detection using the most advanced AI algorithm called the Deep Belief Networks (DBN). They examine the qualities of bogus websites and suggest a pair of attributes for online phishing detection: original characteristics and interactive feature, in addition to train DBN and get the required detection parameters in a small data set. Finally, they employ a large data set to evaluate DBN, and the TPR (True Positive Rate) is around 90%. (Yun Lin et. al, 2021) To begin, describe the technological limits in visual evaluations that prevent present methods being effective (with high correctness) as well as effective (with low runtime overhead) sufficient to be deployed in practise. Then they propose a hybrid system based on Deep Learning called Phishpedia that shows the better accuracy of deep neural networks. (Subhash et. al, 2020) provides a novel method for detecting phishing attacks by combining LSTM and CNN deep neural networks using the use of URLs and HTML pages. Their research involves two distinct kinds of features. URL and HTML functionalities are both available. To remember the URLs, an LSTM

network with 1D convolution is utilised, and second 1D convolutional network is used to recall the HTML features. To obtain the suggested model, these two network underwent training separately and then integrated using a layer called sigmoid by removing the final layer of each model. The model they use has a 98.34 percent accuracy rate. (Roy et. al, 2022) in explored the possibilities of Variants of LSTM models. They used a Kaggle dataset with 450,176 URLs, 345,738 legal and 104,438 phishing URLs. This huge dataset is passed through a series of preprocessing steps. Tokenization and embedding methods are used to convert URL characters to integers, which are then sent to system levels. Three models were trained and ranked by metrics: Bi-LSTM, LSTM, and GRU.

# 3 METHODOLOGY

This section discusses the model utilised for identifying fraudulent websites, as well as information gathering, procedures, and metrics used to evaluate the model.

The dataset is prepared using web scraping various websites such as www.kaggle.com and www.phishtank.com. To build the final dataset, the data is preprocessed and features extracted. The generated data set will be utilised for training the algorithm for machine learning, and its finished version is stored as a pickle file for future use.

The problem here is a binary classification machine learning problem. The machine learning model has to input the URL and predict whether it is a fraudulent URL or a original URL. There are many ways to do this such as detection from text content, from website components, etc. But URL-based detection proved to be more efficient, effective and accurate. To train a ML model for detecting Legitimate and phishing URLs from a set of unknown URLs, we have to collect a number of such URLs in both phishing and Legitimate categories.

Framing an ML problem:

Input : Website URL

Output : Phishing OR Legitimate.

Input → text-related features of a URL (eg: length of URL, presence of @ symbol, etc) →ML model → Prediction

Machine Learning task: Supervised Learning.

Binary classification task – (Two classes to predict, legitimate and phishing)

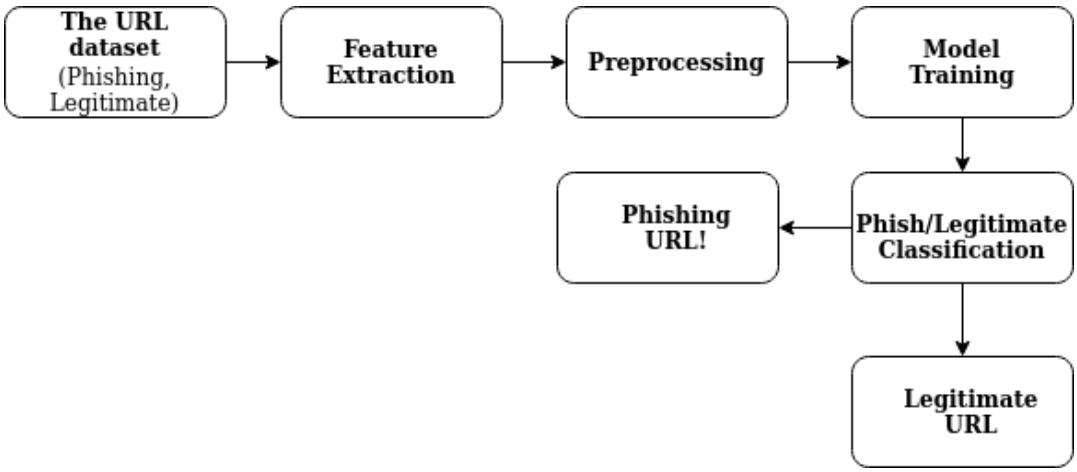



***Figure 3.1 Data Flow Diagram***

The steps involved in the process are:

### 3.1 Data Collection:

URLs of legitimate and phishing websites are collected for ML model training. The phish URLs are collected from the open-source reservoir of www.phishtank.com. They have listed in their website as verified phishes. Out of the two classes of URLs, we need to collect the phishing URLs live. That mean the URLs should be online. We can collect the legitimate URLs from any other old sources.

The legitimate URLs are collected from Cyber Security research of University of New Brunswick (https://www.unb.ca/cic/datasets/url-2016.html).

### 3.2 Preprocess and clean the data for ML model training:

Recognise and deal with missing values in the dataset. This can be accomplished by either eliminating missing value rows or imputing missing values using the mean or median. we change the dataset csv to excel. Then, because certain records in the phishing column were missing, we filled them with NaN values(Geyik et. al, 2021). We detected and deleted distinct numbers. Empty rows are removed and normalised.

### 3.3 Feature extraction :

Features from the URLs can be extracted in many ways. Here in this three types of features have been extracted. A total of 31 features have been extracted from the three type of features.

1. Address bar-based features.
2. Domain-based features.
3. HTML and Javascript-based features.

**1) Address bar-based features**: Address bar-related factors are critical in detecting phishing URLs since they provide significant insights into the structure and properties of the web address. These characteristics are derived from the URL displayed in the address bar of a web browser when a user visits a website. Some popular address bar-related features utilised in phishing detection include(E. Sandi et. al, 2019) :

1. Domain of URL : Obtaining the internet domain address from its URL aids in finding the website's primary address. Phishers frequently utilise false domain names that closely resemble legitimate websites to confuse users.

2. IP Address in URL: Examining the IP address associated with the URL can offer additional details about the website's validity. If the domain section of the URL includes an IP address, the value for that attribute is 1; contrary, it is 0.

3. "@" Symbol in URL : @ Symbol is a frequent presence in the Phishy URLs If the URL has '@' symbol, otherwise, the value given this particular feature is 1.

4. Length of URL : Long URLs are frequently employed in scams to conceal the suspicious material in the address bar. As a result, we've set a length limit of 54 characters. If the URL length is greater than 54, the numerical value allocated to this characteristic is 1, else it is 0.

5. Depth of URL : According to the presence of a '/', this feature determines the amount of subsection pages in the provided url.

6. Redirection "//" in URL : The existence of '//' within the URL indicates that the individual is about to be redirected to another location. However, // is commonly placed sixth in HTTP and seventh in HTTPS. As a result, we notice the occurrence of // in places following the sixth or seventh. If the "//" appears throughout the URL other than following the protocol, the number attributed for this feature is 1; otherwise, it is 0.

7. "http/https" in Domain name: For the purpose of misleading users, scammers may add the "HTTPS" certificate to the domain part of a URL. If the web address has "http/https" in the domain name part, the numerical value assigned this particular feature is 1; otherwise, the value is 0.

8. URL Shortening Services “TinyURL” : A URL shortening service takes a large URL as input and provides a shorter version. Users are sent to the original long URL when they click upon the shortened link. This is performed by using a "HTTP Redirect" on a short domain name, which leads to a webpage with a lengthy URL. If the URL uses Shortening Services, the integer provided to this feature is 1; otherwise, the value is 0.

9. Prefix or Suffix "-" in Domain: If the URL contains the '-' symbol within the web address part that makes up the URL, the value that is assigned to this feature is 1; otherwise, the value is 0.

**2) Domain-Based Features:**

Domain-based features are related to the domain, the date of purchase and the date of expiration of the domain, owner of the domain, etc. This is done with the help of the python-whois library which is a library to extract the details of a domain in python.

1. DNS Record If the name server record is blank or not discovered, the value given to this attribute is 1; otherwise, it is 0.

2. Website Traffic : The amount of people who visit phishing websites via false or fake URLs. Cybercriminals use phishing attacks to construct phoney websites that look like authentic ones in order to deceive users into providing confidential information such as access passwords, financial

information, or personal data. If the domain's rank is one million, the outcome of this attribute is 1, else it is 0.

3. Age of the domain: Typically, phishing websites only exist for a short time. the amount of time a specific domain (website address) is currently registered and operating on the internet. It is a measurement of the amount of time a website has been online since its owner initially registered it. As a result, we consider 6 months to be the youngest age of a legal domain. Age in this context refers to the time between creation and expiration. If the domain is older than six months, the current value of this characteristic is 1, otherwise it is 0.

4. End Period of Domain: The date on which a domain registration expires. When a website is registered, it is normally for a set length of time, usually a year or more. To keep the domain active after the registration time, the domain owner must renew it. A valid domain is one that has been in use for six months or fewer. If the domain's end term is greater than 6 months, the default setting of this characteristic is 1, otherwise it is 0.

**3) HTML-based features**

It is used to examine the structure and content of web pages. These characteristics are taken from the webpage's HTML source code and provide vital information into the website's validity and potential phishing characteristics.This is done with the help of various python libraries such as beautiful soup.

1. IFrame Redirection : IFrame is a type of HTML tag that displays an extra webpage within the currently displayed page. Phishers can hide the HTML elements of the supplementary page by using the "iframe" tag. If the iframe is blank or no response is detected, the integer associated to this feature is 1; otherwise, it is 0.

2. Status Bar Customization : The browser status bar, commonly known as the address bar, enables you to display valuable data or messages to clients as they navigate your website. However, due to safety and user experience considerations, current browsers have limited functionality for customising the status bar. To extract this function, we must examine the website's source code, specifically the "onMouseOver" event, and see if it modifies the status bar. If the answer is empty or there is no mouseover, the numerical value related to this attribute is 1; otherwise, it is 0.

3. Disabling Right Click : Some websites employ this strategy to avoid users from copying material or reading page source, generally in order to protect their intellectual property. Disabling right-click, on the other hand, might be inconvenient for users and may impair their browsing experience. We'll look at the source code of the webpage for the event "event.button==2" to see if

right clicking is disabled. If the response is empty or there is no onmouseover, the numerical value assigned to the aforementioned feature is 1 or 0.

4. Website Forwarding : Website forwarding is a mechanism for redirecting customers from a single web address to another. It is also known as URL sending or URL redirection. It is typically used when a website's domain name changes or when a single page or resource moves to a new address. At least four occasions, phishing websites were successfully routed (Mahajan et al, 2018).

**Additional Features**

1. Presence of subdomain : Check presence of subdomain by checking '.' If present this feature is given value -1, else 1.

2. Favicon: Check the HTML script for favicon -1 if present, value = 1 if not present

3. Anchor URL : Check presence of redirecting anchor URL If present this feature is given value -1, else 1

4. Link in Script tag: Check links in the HTML script .-1 if present, value = 1 if not present

5. Server form handler : Check the presence of server form handler in HTML -1 if present, value = 1 if not present.

6. Info Email in HTML: Check presence info email in the HTML script If present this feature is given value 1, else -1.

7. Abnormal URL: Check response to check abnormal URL .-1 if present, value = 1 if not present.

8. Using pop up window: It is a web development strategy that displays additional content, information, or actions in a second browser window or dialogue box. Popups can be used for a variety of purposes, including displaying alerts, notifications, login forms, and adverts.

9. Page Rank : Global page rank from www.checkpagerank.org -1 if the rank is greater than 100000.

10. Global index: Check presence in google index If present this feature is given value 1, else -1.

11. Links pointing to page: check hyperlinks in the script .-1 if present, value = 1 if not present

12. Statistical report

13. HTTPS protection : HTTPS ensures that the data transmitted between the user and the website is private, secure, and untampered with.

14. Request URL : It is a term used in web development and web security to refer to the Uniform Resource Locator (URL) of a specific resource or webpage that a user's web browser has requested.

### 3.4 Train Machine Learning Models:

Divide the entire set onto Train and Test data sets. This in Usually split in 80:20 or 70:30. Here it is used in the ratio 80:20.

Train = 80% and Test = 20%

The train data shall be used to correctly learn the ML model, while the test data will be employed to assess the model's performance on unknown data.

To assess the accuracy of various strategies, a model is created that employs five distinct algorithms, including Logistic Regression, K Nearest Neighbors Classifier, Random Forest, Decision Tree, SVM, and XGBoost.

### 3.4.1 Model 1: K-Nearest Neighbor Algorithm (KNN)

The K-Nearest Neighbours (KNN) technique is a supervised machine learning approach that can be utilised for regression as well as classification applications. KNN can be used as a method of classification in the field of phishing detection to evaluate whether a given URL is real or phishing based on its attributes.

The KNN algorithm uses a non-parametric, lazy learner approach to classify new data points based on similarity to existing cases. During categorization, it stores all accessible data and executes actions on the dataset. Unlike other methods, it does not immediately acquire information from the initial data set and saves the dataset for subsequent classification.

Assume two different groups, A and B, and we obtain an additional data point x1. Determine which of the following categories this data piece belongs to. A K-NN algorithm is required to address this type of problem. We can simply identify the group of data or class of a certain dataset using K-NN. Consider the diagram below(M Agarwal et. al, 2023):

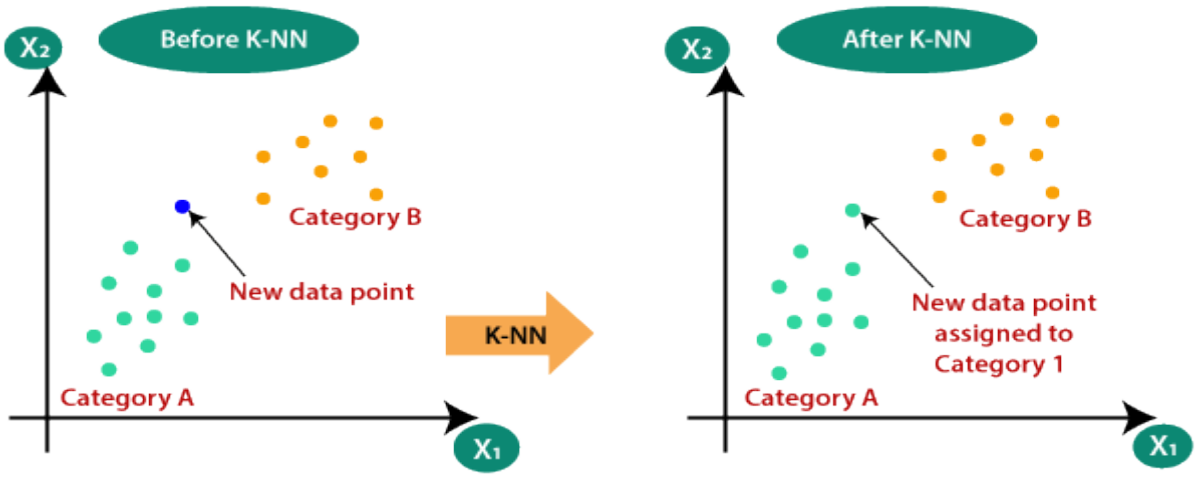



*Figure 3.2: KNN explanation*

### 3.4.2 Model 2: Decision Tree Algorithm

A decision tree is a popular supervised machine learning method that can be utilised in classification and regression. It produces a structure resembling a tree, with every internal node reflecting a feature, each branch indicating a decision based on that feature, and each leaf node representing both a class label (in classification) or a number (in regression). Depending on the kind of tree (ID3, C4.5, CART), the decision tree method recursively divides the data into subgroups based on the feature that gives the best information gain or Gini impurity.

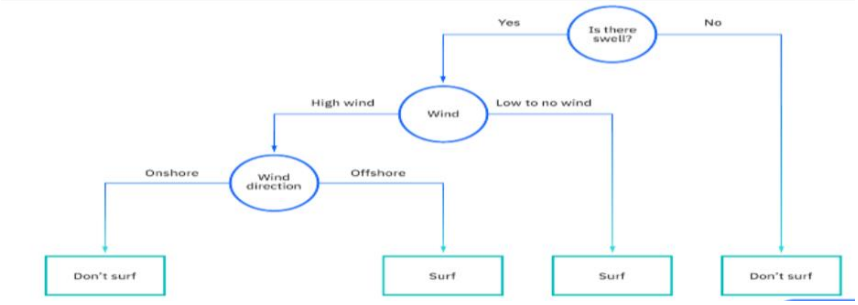



***Figure 3.3: Decision Tree***

A decision tree, as seen in Figure 3.3, starts with a root node with no incoming branches. Outgoing branches from the root node feed into internal nodes known as decision nodes. Both node types analyse available features to produce homogeneous subsets that can be recognised by leaf or terminal nodes. The edges of nodes with leaves reflect all of the possible outcomes of the dataset. (M. Agarwal et al, 2023).

### 3.4.3 Model 3: Random Forest Algorithm

Random Forest is a decision tree-based ensemble learning system. It is noted for its excellent accuracy, robustness, and capacity to handle complicated datasets and is used for regression as well as classification issues. During training, Random Forest creates many decision trees and integrates their predictions to obtain a more accurate and consistent output. The Random Forest algorithm operates in two stages.

1. Creating the Forest:

The algorithm begins by selecting a portion of the initial data at random (with replacement). This subset is referred to as a bootstrap sample. It then builds a decision tree using the bootstrap sample, but only takes into account a random subset of features at each split, generally (total number of characteristics) or log2(total number of features). The process continues to generate several decision trees, each with its own collection of data and attributes.

2. Predictions Voting:

Each decision tree in the forest offers its forecast to make an estimate for a new instance. The ultimate prediction for classification problems is the majority of the vote (mode) of each of the tree forecasts. The final prediction for regression problems is the median average of all unique tree forecasts(E Sandi et. al, 2019).

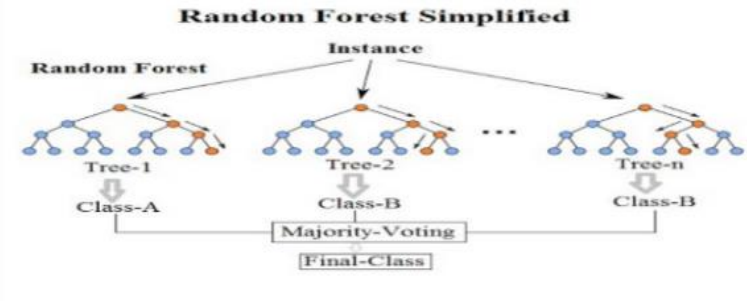



***Figure 3.4: Random forest architecture***

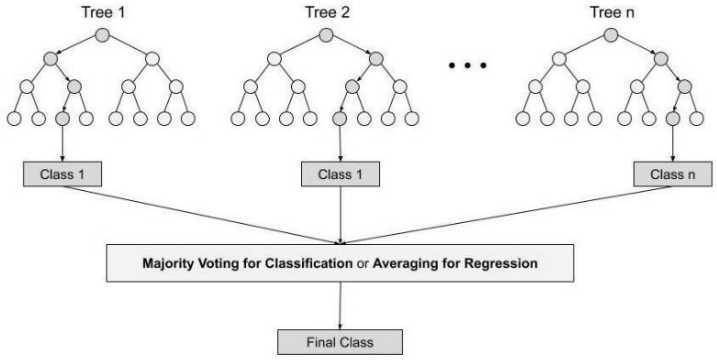



***Figure 3.5: Random forest classifier procedures***.

n Random Forests, important hyperparameters are used to either improve model performance and predictive power or to make the model faster. The following hyperparameters improve predictive power:

1. n_estimators- the number of trees constructed by the algorithm prior to averaging the predictions.

2. max_features- Random forest considers the most features while breaking down a node.

3. mini_sample_leaf- The least number of leaves required to divide an internal node is calculated.

The following hyperparameters boost the speed:

1. n_jobs- tells the engine how many processors it has available. If the number given is 1, a single processor may be used; if the value is -1, no constraint is imposed.

2. The randomization of the sample is controlled by random_state. If the model contains a certain degree of random state and is fed with identical variables as well as training data, it will consistently return the same results.
3. oob_score - The term "out of the bag" is an acronym for "out from the bag." The woodland is haphazard.

### 3.4.4 Model 4: Support Vector Machine Classifier (SVM)

SVM is a supervised machine learning technique that is used in classification and regression applications. It operates by determining the appropriate hyperplane for separating various categories in the data.
For binary classification, the Support Vector Machine technique is used:
Separation along a straight line: SVM starts with the assumption that the information can be partitioned linearly in the space of features by a hyperplane. The boundary of a decision that maximises the margin (distance) among the nearest data points of distinct classes is defined as the hyperplane. These data points are referred to as support vectors.
Soft Margin and Margin: The distance among the decision border and the support vectors is defined as the margin. SVM tries to maximise this margin by providing superior generalisation to previously unseen data. SVM allows for some incorrect classifications in circumstances where data points are not fully separated by introducing a "soft margin." The soft margin enables a more adaptable decision boundary.
Kernel Hack: When the data is not linearly separate in the initial feature space, SVM can use a kernel function to transform it into a higher-dimensional space.
The linear kernel, which is used (for linearly separable data), quadratic kernel, the radial base function (RBF) kernel, and gaussian kernel are all common kernel functions.
Optimization: SVM approaches the issue as a convex optimisation challenge, aiming to discover the best hyperplane that maximises the margin while minimising misclassification errors (E. Sandi et. al, 2019).

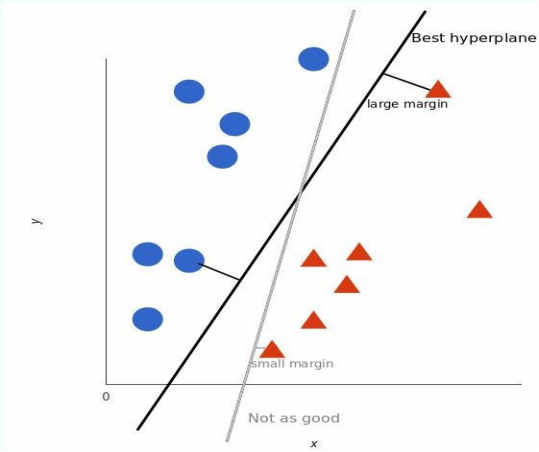



*Figure 3.6 : SVM*

### 3.4.5 Model 5: Logistic Regression

Logistic Regression is a form of statistics used to predict one of two probable outcomes in binary classification situations (e.g., Yes/No, True/False, 1/0). It is a form of Generalised Linear Model (GLM) that estimates the probability of the target variable adhering to each class in order to model the connection between the variables that are independent (features) with the binary dependent variable (target).

The logistic function (sigmoid function) transforms the dependent variable in logistic regression, mapping real-valued expectations to probabilities between 0 and 1. As a linear combination of the feature values, the framework estimates the log-odds of the variable of interest being in the positive class (1) and then uses the logistical equation to obtain the probability(Abuthaha et. al 2021).

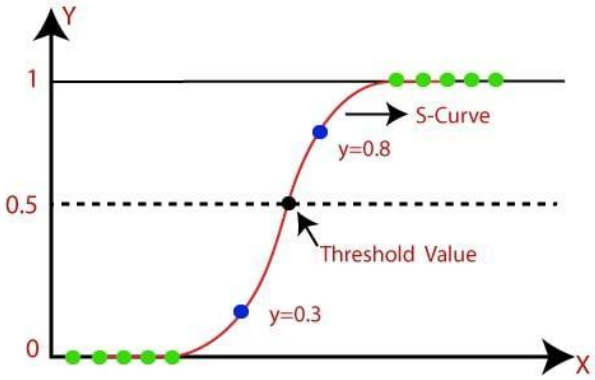



*Figure 3.7: Logistic regression*

A threshold (typically 0.5) is added to the probabilities in order to make predictions. If P(y=1|x) is greater than the threshold, the sample is assigned to the positive class (1); otherwise, it is assigned to the negative class (0).

The logistic regression model is trained using optimisation approaches to determine the best weights (coefficients) for the observed data. The most commonly used optimisation approach is known as "maximum likelihood estimation."

Logistic Regression is widely utilised in a wide range of applications, including problems with binary classification in machine learning, medical diagnosis, marketing, and social sciences. It is straightforward to read and implement, and it can provide insights into the link between the characteristics and the target variable.

### 3.4.6 Model 6: XGBoost Classifier

XGBoost (Extreme Gradient Boosting) is a powerful and strong machine learning algorithm that may be used for categorization as well as analysis. It is a method of ensemble learning which brings together the predictions of numerous weak learners (often decision trees) to produce a more powerful and accurate model.

The basic concept of XGBoost is gradient boosting, which refers to a sequential process that generates several decision trees and progressively corrects the faults committed by prior trees. The algorithm operates like this:

1. Begin the model with an easy base estimator (often a decision tree with one node).
2. Calculate the remainders (the differences between the actual targets and the projected values) after fitting the model to the training data.
3. Try to anticipate the remaining errors by fitting a new estimator to the residuals.
4. To avoid overfitting, update the model by including the new estimates to the ensemble at a low learning rate.
5. Steps 2–4 should be repeated for a predetermined number of iterations (trees), with the weights of each estimator adjusted at each step to minimise the overall error.

# 4 REQUIREMENTS

## 4.1 SYSTEM SPECIFICATION

### 4.1.1 HARDWARE SPECIFICATION

Processor: Intel i5 or higher for training.

RAM: 4 GB or higher.

Graphic card required: Yes (For training)

### 4.1.2 SOFTWARE SPECIFICATION

Operating System : Windows 10

IDE : Google Colab/Jupyter Notebook

Language used : Python 3.7

### 4.1.3 LIBRARIES AND FRAMEWORKS USED

- **Pandas** - python library for dealing with tabular data/dataframe.
- **Numpy** - python library for numerical calculations.
- **Matplotlib** - python library for plotting and visualization.
- **Re** - (regular expression) python library for dealing with strings and texts
- **Os** - python library for os-related operations, file reading and writing
- **Sci-kit learn** - python library for ML algorithms.
- **Pickle** - python library for saving high dimensional data (here, model)
- **Whois** - library to find DNS record of URLs
- **Urllib** - Library to deal with URL strings.
- **Ipaddress** - Python IP address details library
- **BeautifulSoup** - Python library called beautiful soup for HTML related operations
- **datetime** - For the real date and time based operations
- **Requests** - python HTML requests library
- **Googlesearch librar**y -Python library for searching Google, easily.
- **Sklearn.metrics** - for ML model evaluation metrics.
- **xgboost.XGBClassifier** - For XGBoost algorithm.
- **Sklearn.metrics.accuracy_score** - for ML model evaluation metrics.

- **Seaborn library** - python library for graphs and visualizations.
- **Train_test_split - sklearn module** for splitting the dataset into train and test

## 4.2 DATASET COLLECTION

URLs of legitimate and phishing websites are collected for ML model training. The phish URLs are collected from the open-source reservoir of www.phishtank.com. They have listed in their website, verified phishes.

The legitimate URLs are collected from Cyber Security research of University of New Brunswick (https://www.unb.ca/cic/datasets/url-2016.html).

# 5 ANALYSIS AND RESULTS

The following assessment matrix is used to assess and contrast the performance of Machine Learning models: Accuracy, precision, recall, and f - score are all metrics. The percentage of correctly predicted websites is referred to as accuracy. Precision is the proportion of phishing websites that are accurately identified. The number of phishing URLs discovered by the model is counted in the recall metric.

A confusion matrix is a technique for evaluating the correctness of a model developed using machine learning in binary classification situations. It compares the actual target values to the projected values and displays the number of classifications as true positive (TP), true negative (TN), false positive (FP), and false negative (FN). (Masurkar et. al, 2020).

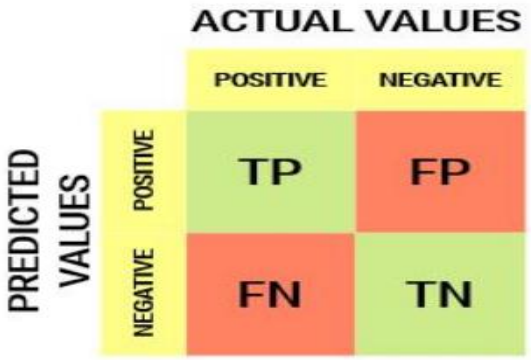



***Figure 5.1: Confusion matrix***

The prediction summary is represented as a matrix in a confusion matrix. It shows the number of accurate and incorrect guesses for each class. It helps to understand the classes that the model misidentifies as other classes (Masurkar et al, 2020).

A TP (True Positive) event occurs when a classification model correctly detects a web page as phishing attacks, whereas a TN (True Negative) incident occurs when a website is incorrectly labelled as Valid URL. An FP (False Positive) occurs when a website is improperly identified as phishing, whereas a FN (False Negative) occurs when the algorithm inaccurately classifies a website as a valid URL when it is in reality phishing. The mathematical equations for the performance metrics are provided below.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP+FN}$$

$$F1\ score = \frac{2*Precision*Recall}{Precision+Recall}$$

The table displays the performance metrics (accuracy, f1 score, recall, and precision) of various machine learning models when applied to a given job. Each model is evaluated based on its ability to accurately categorise cases, with higher numbers indicating better performance.

The Machine Learning models such as KNN (K-Nearest Neighbour), Support Vector Machine(SVM), Logistic Regression, Decision Tree, Random Forest, and XGBoost are implemented. They are evaluated based on evaluation parameters such as accuracy, precision, recall and F1score. The trained models for phishing detection are saved as pickle files and will be tested.

### 5.1 Model 1: K- Nearest Neighbor Model

For both the training and test datasets, the accuracy, f1-score, recall, and precision of the K-Nearest Neighbours (KNN) model's performance are computed and displayed. The accuracy of the KNN model on the training data. The genuine labels of the training data are represented by y_train, while the predicted labels of the KNN model on the training data are represented by y_train_knn. The metrics module's accuracy_score function is used to calculate the accuracy and is depicted below.

```
#computing the accuracy,f1_score,Recall,precision of the model performance

acc_train_knn = metrics.accuracy_score(y_train,y_train_knn)
acc_test_knn = metrics.accuracy_score(y_test,y_test_knn)
print("K-Nearest Neighbors : Accuracy on training Data: {:.3f}".format(acc_train_knn))
print("K-Nearest Neighbors : Accuracy on test Data: {:.3f}".format(acc_test_knn))
print()

f1_score_train_knn = metrics.f1_score(y_train,y_train_knn)
f1_score_test_knn = metrics.f1_score(y_test,y_test_knn)
print("K-Nearest Neighbors : f1_score on training Data: {:.3f}".format(f1_score_train_knn))
print("K-Nearest Neighbors : f1_score on test Data: {:.3f}".format(f1_score_test_knn))
print()

recall_score_train_knn = metrics.recall_score(y_train,y_train_knn)
recall_score_test_knn = metrics.recall_score(y_test,y_test_knn)
print("K-Nearest Neighbors : Recall on training Data: {:.3f}".format(recall_score_train_knn))
print(" K-Nearest Neighbors: Recall on test Data: {:.3f}".format(recall_score_test_knn))
print()

precision_score_train_knn = metrics.precision_score(y_train,y_train_knn)
precision_score_test_knn = metrics.precision_score(y_test,y_test_knn)
print("K-Nearest Neighbors : precision on training Data: {:.3f}".format(precision_score_train_knn))
print("K-Nearest Neighbors : precision on test Data: {:.3f}".format(precision_score_test_knn))
```

```
K-Nearest Neighbors : Accuracy on training Data: 0.989
K-Nearest Neighbors : Accuracy on test Data: 0.955

K-Nearest Neighbors : f1_score on training Data: 0.990
K-Nearest Neighbors : f1_score on test Data: 0.960

K-Nearest Neighbors : Recall on training Data: 0.990
Logistic Regression : Recall on test Data: 0.962

K-Nearest Neighbors : precision on training Data: 0.990
K-Nearest Neighbors : precision on test Data: 0.957
```

***Figure 5.2 Performance Metrics of K-NN***

The accuracy of the KNN model on training data is 98.9% and 95.5% on test data. The F1-score is a statistic that takes precision and recall into account. The F1-score of the KNN model on training data is 0.990 and 0.960 on test data, demonstrating that it can efficiently balance precision and recall. The recall of the KNN model on training data is 0.990 and 0.962 on test data. The precision of the KNN model on training data is 0.990 and 0.957 on test data.

## 5.2 Model 2: Decision Tree

The Decision Tree model's performance metrics are computed using the actual target labels (y_train and y_test) and the predicted labels (y_train_tree and y_test_tree) from the model's predictions.

```
#predicting the target value from the model for the samples

y_train_tree = tree.predict(X_train)
y_test_tree = tree.predict(X_test)
```

```
[ ] #computing the accuracy, f1_score, Recall, precision of the model performance

    acc_train_tree = metrics.accuracy_score(y_train,y_train_tree)
    acc_test_tree = metrics.accuracy_score(y_test,y_test_tree)
    print("Decision Tree : Accuracy on training Data: {:.3f}".format(acc_train_tree))
    print("Decision Tree : Accuracy on test Data: {:.3f}".format(acc_test_tree))
    print()

    f1_score_train_tree = metrics.f1_score(y_train,y_train_tree)
    f1_score_test_tree = metrics.f1_score(y_test,y_test_tree)
    print("Decision Tree : f1_score on training Data: {:.3f}".format(f1_score_train_tree))
    print("Decision Tree : f1_score on test Data: {:.3f}".format(f1_score_test_tree))
    print()

    recall_score_train_tree = metrics.recall_score(y_train,y_train_tree)
    recall_score_test_tree = metrics.recall_score(y_test,y_test_tree)
    print("Decision Tree : Recall on training Data: {:.3f}".format(recall_score_train_tree))
    print("Decision Tree : Recall on test Data: {:.3f}".format(recall_score_test_tree))
    print()

    precision_score_train_tree = metrics.precision_score(y_train,y_train_tree)
    precision_score_test_tree = metrics.precision_score(y_test,y_test_tree)
    print("Decision Tree : precision on training Data: {:.3f}".format(precision_score_train_tree))
    print("Decision Tree : precision on test Data: {:.3f}".format(precision_score_test_tree))
```

```
Decision Tree : Accuracy on training Data: 0.991
Decision Tree : Accuracy on test Data: 0.957

Decision Tree : f1_score on training Data: 0.992
Decision Tree : f1_score on test Data: 0.961

Decision Tree : Recall on training Data: 0.992
Decision Tree : Recall on test Data: 0.962

Decision Tree : precision on training Data: 0.993
Decision Tree : precision on test Data: 0.960
```

***Figure 5.3 Performance metrics of Decision Tree***

The Decision Tree model's accuracy on the training data is 0.991, which means it correctly predicts 99.1% of the cases in the training data. The accuracy on the test data is 0.957, meaning that the model correctly predicts around 95.7% of the events in the test dataset. The F1-score of the Decision Tree model on training data is 0.992, while it is 0.961 on test data. The Decision Tree model has a recall of 0.992 on training data and 0.962 on test data. The Decision Tree model has an accuracy of 0.993 on training data and 0.960 on test data.

### 5.3 Model 3: Random Forest

The model makes predictions using features from the training dataset, and the projected target values are saved in y_train_forest. The model, like the training dataset, makes predictions using characteristics from the test dataset, and the projected target values are saved in y_test_forest.

The Random Forest model's predictions (stored in y_train_forest and y_test_forest) are binary labels, with 1 indicating a legal URL and -1 indicating a phishing URL. As seen in code samples, these projected target values are then used to evaluate the model's performance using various metrics including as accuracy, F1-score, recall, and precision.

```
#predicting the target value from the model for the samples
y_train_forest = forest.predict(X_train)
y_test_forest = forest.predict(X_test)
```

```
#computing the accuracy, f1_score, Recall, precision of the model performance

acc_train_forest = metrics.accuracy_score(y_train,y_train_forest)
acc_test_forest = metrics.accuracy_score(y_test,y_test_forest)
print("Random Forest : Accuracy on training Data: {:.3f}".format(acc_train_forest))
print("Random Forest : Accuracy on test Data: {:.3f}".format(acc_test_forest))
print()

f1_score_train_forest = metrics.f1_score(y_train,y_train_forest)
f1_score_test_forest = metrics.f1_score(y_test,y_test_forest)
print("Random Forest : f1_score on training Data: {:.3f}".format(f1_score_train_forest))
print("Random Forest : f1_score on test Data: {:.3f}".format(f1_score_test_forest))
print()

recall_score_train_forest = metrics.recall_score(y_train,y_train_forest)
recall_score_test_forest = metrics.recall_score(y_test,y_test_forest)
print("Random Forest : Recall on training Data: {:.3f}".format(recall_score_train_forest))
print("Random Forest : Recall on test Data: {:.3f}".format(recall_score_test_forest))
print()

precision_score_train_forest = metrics.precision_score(y_train,y_train_forest)
precision_score_test_forest = metrics.precision_score(y_test,y_test_tree)
print("Random Forest : precision on training Data: {:.3f}".format(precision_score_train_forest))
print("Random Forest : precision on test Data: {:.3f}".format(precision_score_test_forest))
```

```
Random Forest : Accuracy on training Data: 0.990
Random Forest : Accuracy on test Data: 0.968

Random Forest : f1_score on training Data: 0.991
Random Forest : f1_score on test Data: 0.971

Random Forest : Recall on training Data: 0.993
Random Forest : Recall on test Data: 0.973

Random Forest : precision on training Data: 0.990
Random Forest : precision on test Data: 0.960
```

***Figure 5.4 Performance metrics of Random Forest***

The model predicts roughly 99% of the occurrences in the training dataset and approximately 96.8% of the instances in the test dataset correctly. The Random Forest model has an F1-score of 0.991 on training data and 0.971 on test data. The Random Forest model has a recall of 0.993 on training data and 0.973 on test data. The Random Forest model has a precision of 0.990 on training data and 0.960 on test data.

## 5.4 Model 4: Support Vector Machine

Using the learned SVM model (svc), it predicts the target values for the training data (X_train). The predictions are saved in y_train_svc. The test data's target values (X_test) are forecasted using the same SVM model (svc). The predictions are saved in the y_test_svc variable.

```
#computing the accuracy, f1_score, Recall, precision of the model performance

acc_train_svc = metrics.accuracy_score(y_train,y_train_svc)
acc_test_svc = metrics.accuracy_score(y_test,y_test_svc)
print("Support Vector Machine : Accuracy on training Data: {:.3f}".format(acc_train_svc))
print("Support Vector Machine : Accuracy on test Data: {:.3f}".format(acc_test_svc))
print()

f1_score_train_svc = metrics.f1_score(y_train,y_train_svc)
f1_score_test_svc = metrics.f1_score(y_test,y_test_svc)
print("Support Vector Machine : f1_score on training Data: {:.3f}".format(f1_score_train_svc))
print("Support Vector Machine : f1_score on test Data: {:.3f}".format(f1_score_test_svc))
print()

recall_score_train_svc = metrics.recall_score(y_train,y_train_svc)
recall_score_test_svc = metrics.recall_score(y_test,y_test_svc)
print("Support Vector Machine : Recall on training Data: {:.3f}".format(recall_score_train_svc))
print("Support Vector Machine : Recall on test Data: {:.3f}".format(recall_score_test_svc))
print()

precision_score_train_svc = metrics.precision_score(y_train,y_train_svc)
precision_score_test_svc = metrics.precision_score(y_test,y_test_svc)
print("Support Vector Machine : precision on training Data: {:.3f}".format(precision_score_train_svc))
print("Support Vector Machine : precision on test Data: {:.3f}".format(precision_score_test_svc))
```

```
Support Vector Machine : Accuracy on training Data: 0.969
Support Vector Machine : Accuracy on test Data: 0.956

Support Vector Machine : f1_score on training Data: 0.973
Support Vector Machine : f1_score on test Data: 0.961

Support Vector Machine : Recall on training Data: 0.979
Support Vector Machine : Recall on test Data: 0.967

Support Vector Machine : precision on training Data: 0.966
Support Vector Machine : precision on test Data: 0.955
```

***Figure 5.5 Performance metrics of SVM***

The SVM model had an accuracy of about 0.969 on training data and 0.956 on test data. The F1-score, a precision-recall metric, was roughly 0.973 on the training data and 0.961 on the test data. On the training data, the recall, also known as the true positive rate or sensitivity, was roughly 0.979 and on the test data, it was approximately 0.967. The precision on the training data was roughly 0.966 and 0.955 on the test data.

### 5.5 Model 5: Logistic Regression

The log.predict(X_train) function predicts the target values (phishing or legitimate labels) for samples in the X_train training dataset. The predicted labels are saved in the y_train_log variable. Similarly, the log.predict(X_test) method predicts the target values for the samples in the X_test test dataset. The predicted labels are saved in the y_test_log variable.

```
#predicting the target value from the model for the samples

y_train_log = log.predict(X_train)
y_test_log = log.predict(X_test)
```

```
#computing the accuracy, f1_score, Recall, precision of the model performance

acc_train_log = metrics.accuracy_score(y_train,y_train_log)
acc_test_log = metrics.accuracy_score(y_test,y_test_log)
print("Logistic Regression : Accuracy on training Data: {:.3f}".format(acc_train_log))
print("Logistic Regression : Accuracy on test Data: {:.3f}".format(acc_test_log))
print()

f1_score_train_log = metrics.f1_score(y_train,y_train_log)
f1_score_test_log = metrics.f1_score(y_test,y_test_log)
print("Logistic Regression : f1_score on training Data: {:.3f}".format(f1_score_train_log))
print("Logistic Regression : f1_score on test Data: {:.3f}".format(f1_score_test_log))
print()

recall_score_train_log = metrics.recall_score(y_train,y_train_log)
recall_score_test_log = metrics.recall_score(y_test,y_test_log)
print("Logistic Regression : Recall on training Data: {:.3f}".format(recall_score_train_log))
print("Logistic Regression : Recall on test Data: {:.3f}".format(recall_score_test_log))
print()

precision_score_train_log = metrics.precision_score(y_train,y_train_log)
precision_score_test_log = metrics.precision_score(y_test,y_test_log)
print("Logistic Regression : precision on training Data: {:.3f}".format(precision_score_train_log))
print("Logistic Regression : precision on test Data: {:.3f}".format(precision_score_test_log))
```

```
Logistic Regression : Accuracy on training Data: 0.927
Logistic Regression : Accuracy on test Data: 0.924

Logistic Regression : f1_score on training Data: 0.935
Logistic Regression : f1_score on test Data: 0.932

Logistic Regression : Recall on training Data: 0.944
Logistic Regression : Recall on test Data: 0.940

Logistic Regression : precision on training Data: 0.927
Logistic Regression : precision on test Data: 0.925
```

***Figure 5.6 Performance metrics of Logistic Regression***

The model's accuracy on training data is roughly 92.7%, while it is approximately 92.4% on test data. The model's F1-score on training data is roughly 0.935, and it is approximately 0.932 on test data. The model's recall (also known as sensitivity or true positive rate) on training data is roughly 94.4%, and it is approximately 94.0% on test data. The model's precision on training data is roughly 92.7%, and it is approximately 92.5% on test data.

### 5.6 Model 6: XGBoost

It predicts target values for both the training and test datasets. The predictions are saved in the directories y_train_xgb and y_test_xgb, respectively.

```
#predicting the target value from the model for the samples
y_train_xgb = xgb.predict(X_train)
y_test_xgb = xgb.predict(X_test)
```

```
#computing the accuracy, f1_score, Recall, precision of the model performance

acc_train_xgb = metrics.accuracy_score(y_train,y_train_xgb)
acc_test_xgb = metrics.accuracy_score(y_test,y_test_xgb)
print("XGBoost Classifier : Accuracy on training Data: {:.3f}".format(acc_train_xgb))
print("XGBoost Classifier : Accuracy on test Data: {:.3f}".format(acc_test_xgb))
print()

f1_score_train_xgb = metrics.f1_score(y_train,y_train_xgb)
f1_score_test_xgb = metrics.f1_score(y_test,y_test_xgb)
print("XGBoost Classifier : f1_score on training Data: {:.3f}".format(f1_score_train_xgb))
print("XGBoost Classifier : f1_score on test Data: {:.3f}".format(f1_score_test_xgb))
print()

recall_score_train_xgb = metrics.recall_score(y_train,y_train_xgb)
recall_score_test_xgb = metrics.recall_score(y_test,y_test_xgb)
print("XGBoost Classifier : Recall on training Data: {:.3f}".format(recall_score_train_xgb))
print("XGBoost Classifier : Recall on test Data: {:.3f}".format(recall_score_train_xgb))
print()

precision_score_train_xgb = metrics.precision_score(y_train,y_train_xgb)
precision_score_test_xgb = metrics.precision_score(y_test,y_test_xgb)
print("XGBoost Classifier : precision on training Data: {:.3f}".format(precision_score_train_xgb))
print("XGBoost Classifier : precision on test Data: {:.3f}".format(precision_score_train_xgb))
```

```
XGBoost Classifier : Accuracy on training Data: 0.988
XGBoost Classifier : Accuracy on test Data: 0.965

XGBoost Classifier : f1_score on training Data: 0.989
XGBoost Classifier : f1_score on test Data: 0.969

XGBoost Classifier : Recall on training Data: 0.992
XGBoost Classifier : Recall on test Data: 0.992

XGBoost Classifier : precision on training Data: 0.986
XGBoost Classifier : precision on test Data: 0.986
```

***Figure 5.7 Performance metrics of XGBoost***

The XGBoost Classifier model's accuracy on training data is 0.988 and on test data is 0.965. The F1 score on the training data is 0.989, which is a balanced assessment of the model's precision and recall, and it is 0.969 on the test data. The recall on training data is 0.992 while the recall on test data is 0.992. The precision on training data is 0.986, which indicates that out of all the URLs predicted as phishing, the precision on test data is 0.986.

The below given table compares the performance metrics of all the machine learning models employed in the study.

```
[ ] #Sorting the datafram on accuracy
    sorted_result=result.sort_values(by=['Accuracy', 'f1_score'],ascending=False).reset_index(drop=Tr
```

```
# dispalying total result
sorted_result
```

| | ML Model | Accuracy | f1_score | Recall | Precision |
|---|---|---|---|---|---|
| 0 | Random Forest | 0.968 | 0.971 | 0.993 | 0.990 |
| 1 | XGBoost Classifier | 0.965 | 0.969 | 0.992 | 0.986 |
| 2 | Decision Tree | 0.957 | 0.961 | 0.992 | 0.993 |
| 3 | Support Vector Machine | 0.956 | 0.961 | 0.979 | 0.966 |
| 4 | K-Nearest Neighbors | 0.955 | 0.960 | 0.990 | 0.990 |
| 5 | Logistic Regression | 0.924 | 0.932 | 0.944 | 0.927 |

***Table 5.1 Comparison of ML methods***

According to the results, the Random Forest and XGBoost Classifier models had the highest accuracy, f1 score, recall, and precision of all the models. These algorithms are regarded as high-performing and well-suited to the task of detecting phishing URLs.

# 6 DESIGN

The data flow diagram in the figure shows the process of phishing URL detection in brief. The URL dataset contains URLs in the two classes: phishing and legitimate. This is fed to the preprocessing and feature extraction then to the model training process. The figure 6.1 shows a detailed explanation of the process.

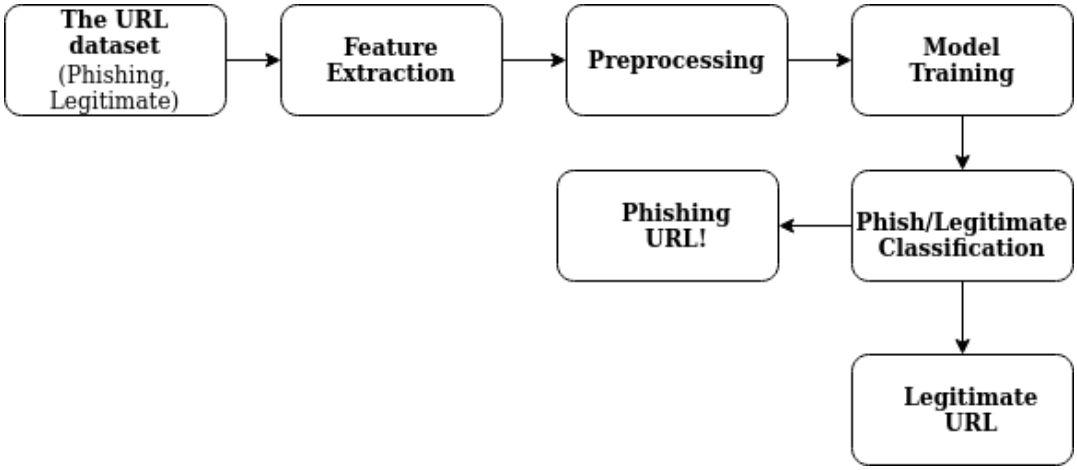



***Figure 6.1 Data Flow Diagram of the System Design***

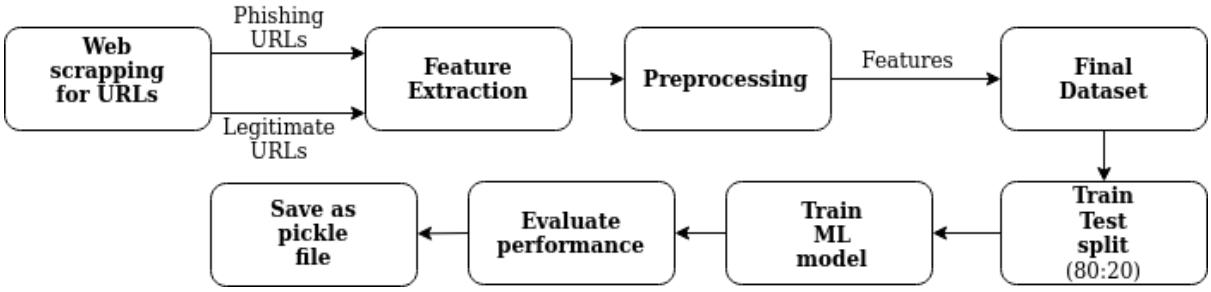



***Figure 6.2: Training and Testing***

The URLs are raw URLs of legitimate and phishing websites. They are converted into a ML-trainable dataset using the preprocessing techniques. The final dataset is split into train and test in 80:20 ratio. The training set is used to train the model, and the remaining 20% is used for testing the model performance for predicting phishing or legitimate from input URLs. The model is saved as a pickle file.

# 7 IMPLEMENTATION

## 7.1 Data Collection

URLs of legitimate and phishing websites are collected for ML model training. The phish URLs are collected from the open-source reservoir of www.phishtank.com. They have listed in their website as verified phishes.

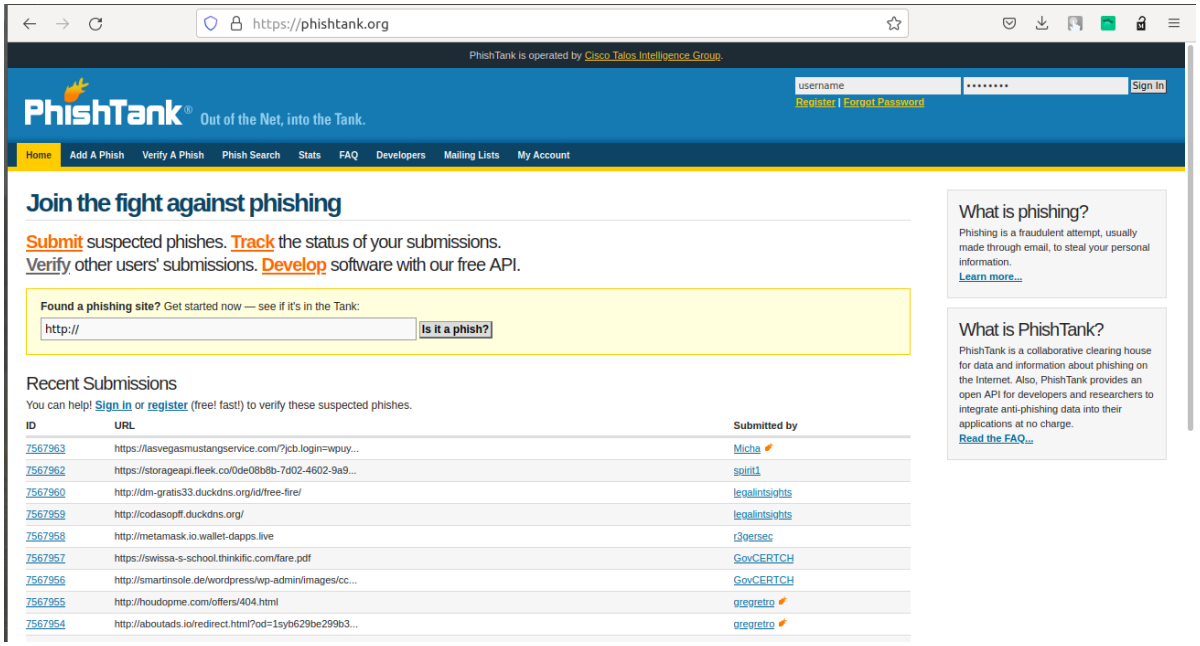



***Figure 7.1 : The phishtank website where the phishing URLs are collected.***

Here, around 78505 URLs have been extracted and restricted them into 35377 by dropping the rest. This is to make the dataset balanced with 35377 phishing and 35377 legitimate URLs.

```
--2022-11-23 14:00:26--  https://data.phishtank.com/data/online-valid.csv
Resolving data.phishtank.com (data.phishtank.com)... failed: Name or service not known.
wget: unable to resolve host address 'data.phishtank.com'
```

```
75]: 1 #loading the phishing URLs data to dataframe
     2 phishdata = pd.read_csv("online-valid.csv")    # read the csv file of phishinbg URLs downloaded
     3 phishdata.head()     # Print the first 5 samples.
```

75]:

| | phish_id | url | phish_detail_url | submission_time | verified | verification_time | online | target |
|---|---|---|---|---|---|---|---|---|
| 0 | 7953552 | https://kvlxjn.webwave.dev/ | http://www.phishtank.com/phish_detail.php?phis... | 2022-11-22T19:01:38+00:00 | yes | 2022-11-22T19:12:58+00:00 | yes | Other |
| 1 | 7953551 | https://one487.weebly.com/ | http://www.phishtank.com/phish_detail.php?phis... | 2022-11-22T19:01:12+00:00 | yes | 2022-11-22T19:12:58+00:00 | yes | Other |
| 2 | 7953549 | https://hy-108491.weeblysite.com/ | http://www.phishtank.com/phish_detail.php?phis... | 2022-11-22T19:01:01+00:00 | yes | 2022-11-22T19:03:56+00:00 | yes | Other |
| 3 | 7953548 | https://74674login.weebly.com/ | http://www.phishtank.com/phish_detail.php?phis... | 2022-11-22T19:00:54+00:00 | yes | 2022-11-22T19:03:56+00:00 | yes | Other |
| 4 | 7953547 | https://h3h556.webwave.dev/ | http://www.phishtank.com/phish_detail.php?phis... | 2022-11-22T19:00:20+00:00 | yes | 2022-11-22T19:03:56+00:00 | yes | Other |

```
76]: 1 phishdata.shape
```

```
76]: (78505, 8)
```

We have 78505 phishing URLs

***Figure 7.2: Phishing URLs collected***

The legitimate URLs are collected from Cyber Security research of University of New Brunswick (https://www.unb.ca/cic/datasets/url-2016.html).

```
In [77]: legitdata.head()    # Print the first 5 samples
```

Out[77]:

| | URLs |
|---|---|
| 0 | http://1337x.to/torrent/1110018/Blackhat-2015-... |
| 1 | http://1337x.to/torrent/1122940/Blackhat-2015-... |
| 2 | http://1337x.to/torrent/1124395/Fast-and-Furio... |
| 3 | http://1337x.to/torrent/1145504/Avengers-Age-o... |
| 4 | http://1337x.to/torrent/1160078/Avengers-age-o... |

```
In [78]: legitdata.shape
Out[78]: (35377, 1)
```

We have 35377 legitimate URLs

***Figure 7.3: Legitimate URLs collected***

So a total 70754 URLs are collected that have 35377 in phishing and 35377 in legitimate.

## 7.2 Feature Extraction

Features from the URLs can be extracted in many ways. Here in this three types of features have been extracted

1. Address bar-based features- The qualities and information displayed in a web browser's address bar are referred to as address bar-related features. These characteristics give critical information about the location, security, and content of the online page. Here 9 features are extracted.
2. Domain-based features - Domain-based features are qualities and attributes derived from a website's domain name. Features such as DNS record, website traffic, age of domain, end period of domain are extracted here.
3. HTML -based features -It refers to traits and qualities retrieved from a web page's HTML (Hypertext Markup Language) source code. These elements provide useful information on the structure, content, and appearance of the web page. 4 feature are extracted here.

After extracting all these 17 features we will have the final dataset that is ready to be used for the model training.

```
# Check the saved dataset

test_17 = pd.read_csv('final_17.csv')
```

```
test_17.head()
```

| | Domain | Have_IP | Have_At | URL_Length | URL_Depth | Redirection | https_Domain | TinyURL | Prefix/Suffix | DNS_Record | Web_Traffic | Domain_Age | Dom |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | graphicriver.net | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | |
| 1 | ecnavi.jp | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | |
| 2 | hubpages.com | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | |
| 3 | extratorrent.cc | 0 | 0 | 1 | 3 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | |
| 4 | icicibank.com | 0 | 0 | 1 | 3 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | |

```
test_17.shape
```

```
(70754, 18)
```

***Figure 7.4: The final dataset***

The final dataset as shown in figure 6 has a total of 10000 samples, 17 features and 1 label.

Analyze the dataset:

70754 Samples : 35377 phishing and 35377 legitimate samples

17 features and One Label: 0 for legitimate and 1 for phishing URLs.

Features: ['Domain', 'Have_IP', 'Have_At', 'URL_Length', 'URL_Depth', 'Redirection', 'https_Domain', 'TinyURL', 'Prefix/Suffix', 'DNS_Record', 'Web_Traffic', 'Domain_Age', 'Domain_End', 'iFrame', 'Mouse_Over', 'Right_Click', 'Web_Forwards"]

Additional features in the context of machine learning for URL-based phishing detection can include numerous properties retrieved from the URL or web page that provide valuable information to the model. These features are used to increase the efficiency of machine learning algorithms and the accuracy of phishing detection. Here 14 additional features are extracted.

```
final_dataset.head()
```

| | Index | UsingIP | LongURL | ShortURL | Symbol@ | Redirecting// | PrefixSuffix- | SubDomains | HTTPS | D |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 1 | 1 | -1 | 0 | 1 | |
| 1 | 1 | 1 | 0 | 1 | 1 | 1 | -1 | -1 | -1 | |
| 2 | 2 | 1 | 0 | 1 | 1 | 1 | -1 | -1 | -1 | |
| 3 | 3 | 1 | 0 | -1 | 1 | 1 | -1 | 1 | 1 | |
| 4 | 4 | -1 | 0 | -1 | 1 | -1 | -1 | 1 | 1 | |

5 rows × 32 columns

```
final_dataset.shape
```

```
(70754, 32)
```

***Figure 7.5: Final dataset with 30 features.***

**Details of the final Dataset:**

In this scenario, a collection of URL samples (70,754 data samples) is employed for phishing detection. The dataset contains 35,377 phishing URLs and 35,377 valid URLs, making it balanced with an equal number of positive (legitimate) and negative (phishing) samples.

The collection is defined by 31 features that are extracted from URLs and online pages. These elements provide useful information about the properties of the URLs and aid in distinguishing between legitimate and phishing URLs.

The dataset has been labelled, with legal URLs assigned a label of 1 and phishing URLs awarded a label of -1. The labels are used to represent the truth and guide the machine learning model during training.

Only 10,000 data samples are selected from the original dataset to simplify the training procedure and reduce computing complexity. This data subset will be utilised to train the machine learning model.

Before beginning training, the dataset is examined for any problems. There are no outliers, which are extreme values that could impact the model's performance. Furthermore, the dataset contains no missing values, indicating that all of the needed attributes are available and full for each data sample.

The absence of outliers and missing numbers benefits the machine learning process. Outliers can bias the model's learning process, resulting in inferior performance. Missing values, if present, would necessitate imputation or data purification, which can be time-consuming and error-prone.

A balanced dataset with an equal number of legal and phishing samples is also advantageous. Imbalanced datasets might generate bias in the model, favouring the majority class (in this case, valid URLs) and resulting in poor performance in recognising the minority class (phishing URLs). The balanced design of the dataset ensures that the model is equally exposed to both classes throughout training.

```
#description of dataset

final_dataset.describe().T
```

| | count | mean | std | min | 25% | 50% | 75% | max |
|---|---|---|---|---|---|---|---|---|
| UsingIP | 10000.0 | 0.3548 | 0.934989 | -1.0 | -1.0 | 1.0 | 1.0 | 1.0 |
| LongURL | 10000.0 | -0.6351 | 0.764465 | -1.0 | -1.0 | -1.0 | -1.0 | 1.0 |
| ShortURL | 10000.0 | 0.7394 | 0.673300 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| Symbol@ | 10000.0 | 0.7100 | 0.704237 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| Redirecting// | 10000.0 | 0.7406 | 0.671980 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| PrefixSuffix- | 10000.0 | -0.7354 | 0.677667 | -1.0 | -1.0 | -1.0 | -1.0 | 1.0 |
| SubDomains | 10000.0 | 0.0381 | 0.812966 | -1.0 | -1.0 | 0.0 | 1.0 | 1.0 |
| HTTPS | 10000.0 | 0.2539 | 0.911218 | -1.0 | -1.0 | 1.0 | 1.0 | 1.0 |
| DomainRegLen | 10000.0 | -0.3388 | 0.940905 | -1.0 | -1.0 | -1.0 | 1.0 | 1.0 |
| Favicon | 10000.0 | 0.6280 | 0.778252 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| NonStdPort | 10000.0 | 0.7266 | 0.687095 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| HTTPSDomainURL | 10000.0 | 0.6752 | 0.737672 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| RequestURL | 10000.0 | 0.1878 | 0.982256 | -1.0 | -1.0 | 1.0 | 1.0 | 1.0 |
| AnchorURL | 10000.0 | -0.0749 | 0.716060 | -1.0 | -1.0 | 0.0 | 0.0 | 1.0 |
| LinksInScriptTags | 10000.0 | -0.1184 | 0.763701 | -1.0 | -1.0 | 0.0 | 0.0 | 1.0 |
| ServerFormHandler | 10000.0 | -0.6031 | 0.758042 | -1.0 | -1.0 | -1.0 | -1.0 | 1.0 |
| InfoEmail | 10000.0 | 0.6344 | 0.773044 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| AbnormalURL | 10000.0 | 0.7046 | 0.709640 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| WebsiteForwarding | 10000.0 | 0.1172 | 0.321675 | 0.0 | 0.0 | 0.0 | 0.0 | 1.0 |
| StatusBarCust | 10000.0 | 0.7618 | 0.647845 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| DisableRightClick | 10000.0 | 0.9144 | 0.404832 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| UsingPopupWindow | 10000.0 | 0.6136 | 0.789657 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| IframeRedirection | 10000.0 | 0.8162 | 0.577798 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| AgeofDomain | 10000.0 | 0.0008 | 1.000050 | -1.0 | -1.0 | 1.0 | 1.0 | 1.0 |
| DNSRecording | 10000.0 | 0.3352 | 0.942194 | -1.0 | -1.0 | 1.0 | 1.0 | 1.0 |
| WebsiteTraffic | 10000.0 | 0.2909 | 0.832794 | -1.0 | 0.0 | 1.0 | 1.0 | 1.0 |
| PageRank | 10000.0 | -0.4648 | 0.885460 | -1.0 | -1.0 | -1.0 | 1.0 | 1.0 |
| GoogleIndex | 10000.0 | 0.7238 | 0.690044 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| LinksPointingToPage | 10000.0 | 0.3437 | 0.569915 | -1.0 | 0.0 | 0.0 | 1.0 | 1.0 |
| StatsReport | 10000.0 | 0.7050 | 0.709243 | -1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| Label | 10000.0 | 0.1128 | 0.993667 | -1.0 | -1.0 | 1.0 | 1.0 | 1.0 |

***Figure7.6 Final Dataset***

In the Figure below corelation heatmap is a graphical depiction of the coefficients of correlation between several features (variables) in a dataset. It helps to discover which elements are significantly linked with each other and which features may have a higher influence on the target variable (legitimate vs. phishing URLs) in the context of the dataset used for phishing detection (Do et. al, 2022).

```
#Correlation heatmap

plt.figure(figsize=(15,15))
sns.heatmap(final_dataset.corr(), annot=True)
plt.show()
```

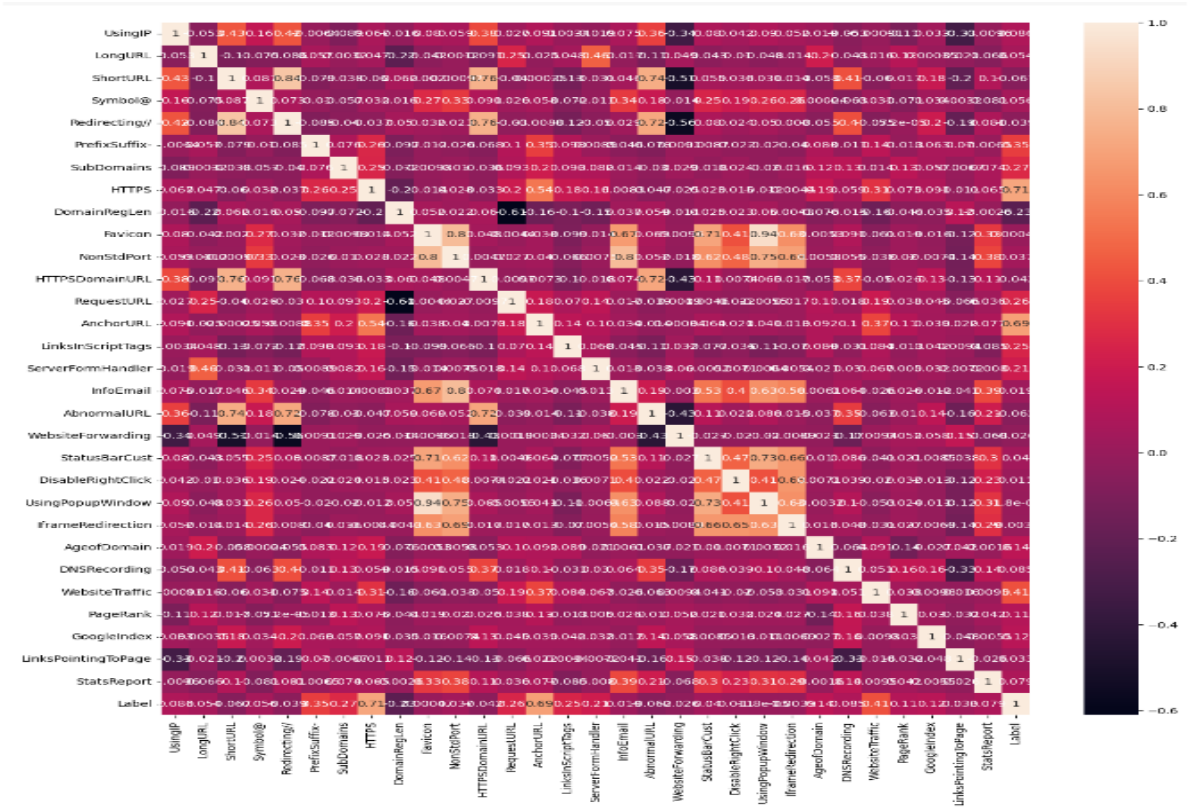



***Figure 7.7 Heatmap Visualization***

The pie chart below depicts the distribution of legitimate and phishing URLs in the sample. It displays the proportion of each type in respect to the total number of URLs in the dataset visually.

### 7.4 Train-Test split the dataset:

Split the dataset into Train and Test data. This in Usually split in 80:20 or 70:30. Here it is used in the ratio 80:20.

Train = 80% and Test = 20%

The train data will be used to properly train the ML model, and the test information will be used to evaluate the model's efficacy on unknown data.

```
[ ] # Splitting the dataset into dependant and independant fetature

    X = final_dataset.drop(["Label"],axis =1)
    y = final_dataset["Label"]
```

```
[ ] # Splitting the dataset into train and test sets: 80-20 split

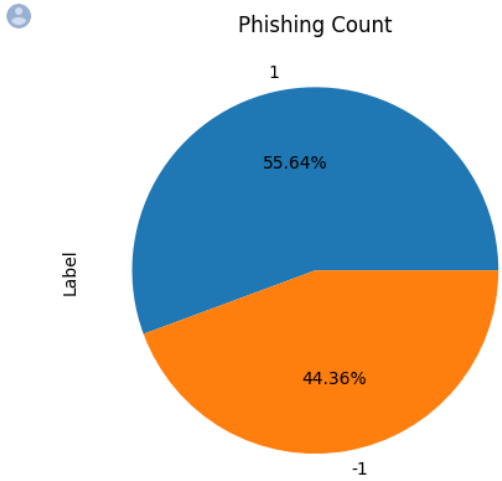
    from sklearn.model_selection import train_test_split

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)
    X_train.shape, y_train.shape, X_test.shape, y_test.shape
```

```
((8000, 30), (8000,), (2000, 30), (2000,))
```

```
# Phishing Count in pie chart

final_dataset['Label'].value_counts().plot(kind='pie',autopct='%1.2f%%')
plt.title("Phishing Count")
plt.show()
```




```
[ ] final_dataset.Label.value_counts()
```

```
 1    5564
-1    4436
Name: Label, dtype: int64
```

***Figure 7.8 Splitting dataset in Train – Test set ratio***

## 7.5 Model Building And Training

### Model 1: K-Nearest Neighbor Algorithm (KNN)

K-Nearest Neighbour is a straightforward Machine Learning methodology that employs the Supervised Learning technique. The method known as K-NN presume resemblance among the new case/data and previous cases and assigns the new case to the category most comparable to the ones that already exist (Buket et. al, 2021).

Construct a K_NeighborsClassifier class with 1 neighbours (the optimal value for your data). The model is then trained using the fit() method with its initial data X_train and the associated labels y_train.

```
# K-Nearest Neighbors Classifier model
from sklearn.neighbors import KNeighborsClassifier

# instantiate the model
knn = KNeighborsClassifier(n_neighbors=1)

# fit the model
knn.fit(X_train,y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier(n_neighbors=1)
```

***Figure 7.9 Screenshot of Training K-NN***

After fitting the model, it is ready to make predictions on new, unseen data using the predict() method:

```
#predicting the target value from the model for the samples


y_train_knn = knn.predict(X_train) # Predict for Train set
y_test_knn = knn.predict(X_test)  # Predict for Test set
```

***Figure 7.10 Screenshot of Prediction***

We generate a KNN model with that 'K' value for each value of 'K' in the range of 1 to 20. On the training set, we train the KNN model, and on the validation set, we make predictions. Using a performance metric such as accuracy, we calculate the accuracy of the KNN model on the validation set.

```
training_accuracy = []
test_accuracy = []
# try max_depth from 1 to 20
depth = range(1,20)
for n in depth:
    knn = KNeighborsClassifier(n_neighbors=n)

    knn.fit(X_train, y_train)
    # record training set accuracy
    training_accuracy.append(knn.score(X_train, y_train))
    # record generalization accuracy
    test_accuracy.append(knn.score(X_test, y_test))


#plotting the training & testing accuracy for n_estimators from 1 to 20
plt.plot(depth, training_accuracy, label="training accuracy")
plt.plot(depth, test_accuracy, label="test accuracy")
plt.ylabel("Accuracy")
plt.xlabel("n_neighbors")
plt.legend();
```

***Figure 7.11 Screenshot of Training Vs Test Accuracy of KNN***

In this code, we generate a KNN model for each number of n_neighbors from 1 to 20 and calculate the training and testing accuracies for each model. We next plot the accuracies versus the number of neighbours to see how the model's performance changes as the number of neighbours increases. The x-axis displays the values of n_neighbors, while the y-axis reflects the matching training and testing accuracies. The graphic will show how the model's accuracy varies with the number of neighbours utilised for categorization as shown in the figure. Both accuracies are high at n = 1.

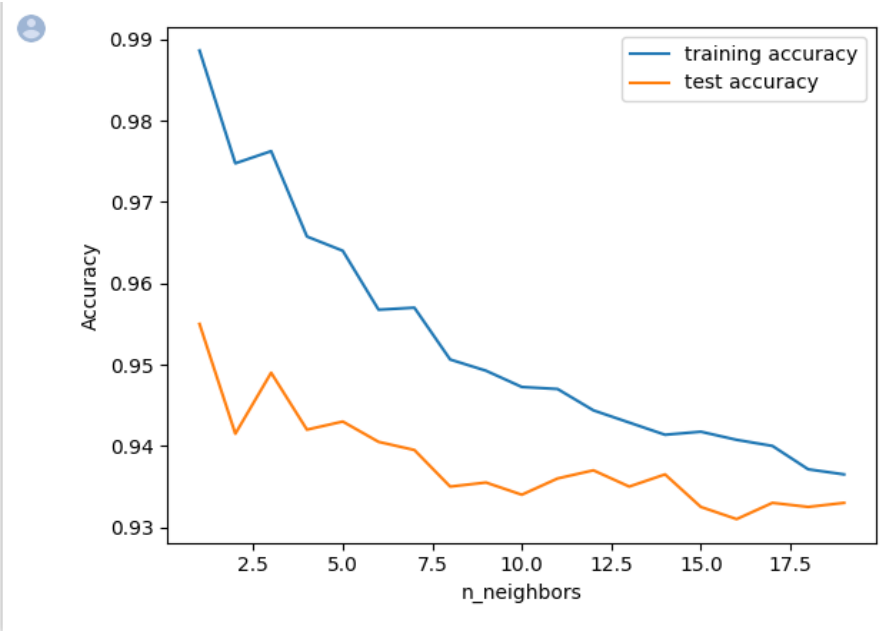



***Figure 7.12 Plot of Training vs Test accuracy***

**Model 2: Decision Tree Algorithm**

Decision Tree is a Supervised method of learning that may be used for classification as well as regression problems, however it is more typically used for classification. (Buket et. al, 2021) It is a tree-structured classification in which core nodes have dataset properties, branches hold decision rules, and every node in the leaf holds the outcome.

```
[ ]  # Decision Tree Classifier model
     from sklearn.tree import DecisionTreeClassifier
```

```
[ ]  training_accuracy = []
     test_accuracy = []
     # try max_depth from 1 to 30
     depth = range(1,30)
     for n in depth:
         tree_test = DecisionTreeClassifier(max_depth=n)

         tree_test.fit(X_train, y_train)
         # record training set accuracy
         training_accuracy.append(tree_test.score(X_train, y_train))
         # record generalization accuracy
         test_accuracy.append(tree_test.score(X_test, y_test))


     #plotting the training & testing accuracy for max_depth from 1 to 30
     plt.plot(depth, training_accuracy, label="training accuracy")
     plt.plot(depth, test_accuracy, label="test accuracy")
     plt.ylabel("Accuracy")
     plt.xlabel("max_depth")
     plt.legend();
```

***Figure 7.13 Screenshot of Training Vs Test Accuracy of Decision Tree***

In this code, we build a Decision Tree model for each max depth value ranging from 1 to 30 and calculate the training and testing accuracies for each model. The accuracies are then shown against the maximum depths to show how the model's performance varies with decision tree depth.
The x-axis depicts maximum depth values, while the y-axis reflects training and testing accuracies. Both accuracies are high at max depth = 30

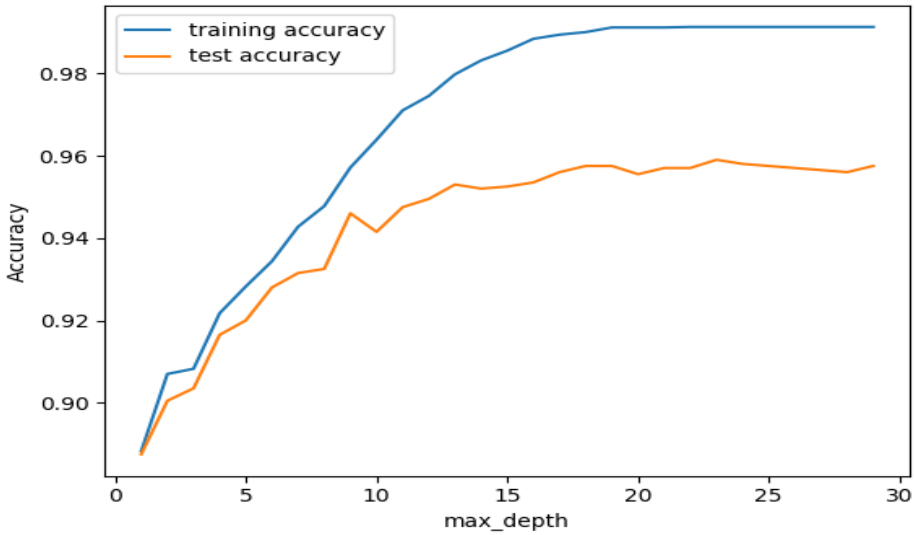



***Figure 7.14 Plot of training Vs Test Accuracy of Decision tree***

**Model 3: Random Forest Algorithm**

Random Forest is a well-known machine learning algorithm from the method of supervised learning. It can be applied to problems with classification as well as regression in machine learning(Buket et. al, 2021). It is built on the notion of collaborative learning, which is a method that involves integrating numerous classifiers to resolve a complex problem and improve the model's performance.

```
[ ]  # Random Forest Classifier Model
     from sklearn.ensemble import RandomForestClassifier
```

```
[ ]  training_accuracy = []
     test_accuracy = []
     # try max_depth from 1 to 20
     depth = range(1,20)
     for n in depth:
         forest_test =  RandomForestClassifier(n_estimators=n)

         forest_test.fit(X_train, y_train)
         # record training set accuracy
         training_accuracy.append(forest_test.score(X_train, y_train))
         # record generalization accuracy
         test_accuracy.append(forest_test.score(X_test, y_test))


     #plotting the training & testing accuracy for n_estimators from 1 to 20
     plt.figure(figsize=None)
     plt.plot(depth, training_accuracy, label="training accuracy")
     plt.plot(depth, test_accuracy, label="test accuracy")
     plt.ylabel("Accuracy")
     plt.xlabel("n_estimators")
     plt.legend();
```

***Figure 7.15 Screenshot of Training Vs Test Accuracy of Random Forest***

We build a Random Forest classifier for each maximum depth value between 1 and 20 and calculate the training and testing accuracies for each model. The accuracies are then shown against the maximum depths to show how the model's performance changes as the depth of the trees in the Random Forest increases.

The x-axis depicts maximum depth values, while the y-axis depicts training and testing accuracies. The plot will show how the model's accuracy changes as the Random Forest becomes more sophisticated. Plotting test and training accuracy are done for number of trees from 1 to 20. Both accuracies are high at number of trees = 10.

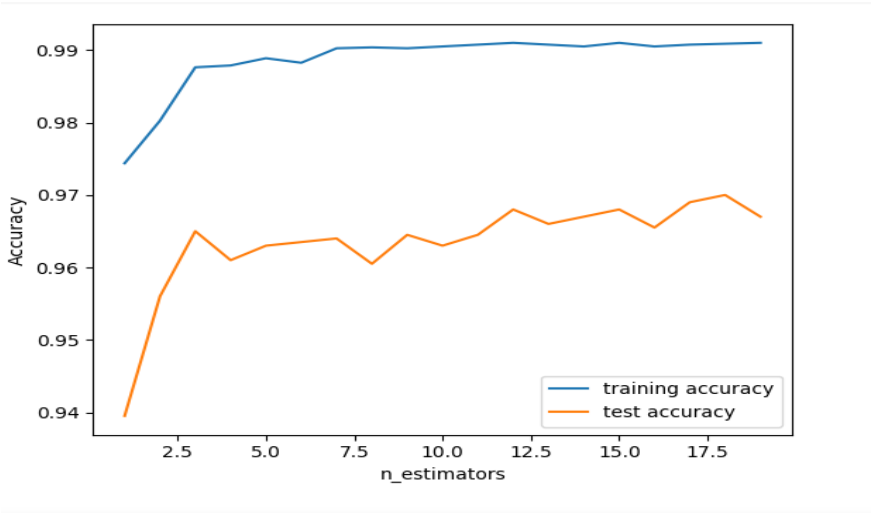



***Figure 7.16 Plot of Training Vs Test Accuracy of Random Forest***

**Model 4: Support Vector Machine Classifier (SVM)**

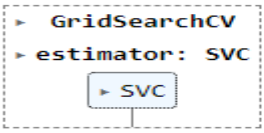
SVM is a strong classification algorithm that is used to distinguish between legal and phishing URLs by locating the optimal hyperplane that optimally divides the two classes.

```
[ ]  # Support Vector Classifier model
     from sklearn.svm import SVC
     from sklearn.model_selection import GridSearchCV
```

```
[ ]  # defining parameter range
     param_grid = {'gamma': [0.1],'kernel': ['rbf','linear']}

     svc = GridSearchCV(SVC(), param_grid)

     # fitting the model for grid search
     svc.fit(X_train, y_train)
```




```
[ ]  #predicting the target value from the model for the samples
     y_train_svc = svc.predict(X_train)
     y_test_svc = svc.predict(X_test)
```

***Figure 7.17 Screenshot of Training Vs Test Accuracy of SVM***

GridSearchCV is used in the provided code to tune the hyperparameters of the Support Vector Machine (SVM) classifier. The purpose is to determine the optimal hyperparameter combination from the given parameter grid (param_grid) to improv
e the SVM model's performance.

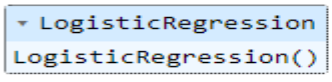
**Model 5: Logistic Regression**

Logistic Regression is used as a baseline approach to evaluate the performance of the project's other complicated models.

```
[ ]  # Linear regression model
     from sklearn.linear_model import LogisticRegression
     #from sklearn.pipeline import Pipeline

     # instantiate the model
     log = LogisticRegression()

     # fit the model
     log.fit(X_train,y_train)
```




```
[ ]  #predicting the target value from the model for the samples

     y_train_log = log.predict(X_train)
     y_test_log = log.predict(X_test)
```

***Figure 7.18 Screenshot of Training Vs Test Accuracy of Logistic Regression***

The model learns the coefficients and intercept that determine the linear relationship between the features (X_train) and the target variable (y_train) during this process. The fitting procedure is based on the maximum likelihood estimation principle, in which the model seeks the parameters that maximise the likelihood of seeing the provided training data. The model is then ready to generate predictions on new data after being fitted.

### Model 6: XGBoost Classifier

XGBoost is a sophisticated boosting technique that combines weak learners sequentially to generate a stronger and more accurate model.

```
!pip install xgboost==1.5
```

```
[ ]  #  XGBoost Classifier Model
     from xgboost import XGBClassifier

     # instantiate the model
     xgb = XGBClassifier()

     # fit the model
     xgb.fit(X_train,y_train)
```

```
[05:25:13] WARNING: ../src/learner.cc:1115: Starting in XGBoost 1.3.0, the default evaluation metric used wit
```

```
XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
              colsample_bynode=1, colsample_bytree=1, enable_categorical=False,
              gamma=0, gpu_id=-1, importance_type=None,
              interaction_constraints='', learning_rate=0.300000012,
              max_delta_step=0, max_depth=6, min_child_weight=1, missing=nan,
              monotone_constraints='()', n_estimators=100, n_jobs=2,
              num_parallel_tree=1, predictor='auto', random_state=0,
              reg_alpha=0, reg_lambda=1, scale_pos_weight=1, subsample=1,
              tree_method='exact', validate_parameters=1, verbosity=None)
```

```
[ ]  #predicting the target value from the model for the samples
     y_train_xgb = xgb.predict(X_train)
     y_test_xgb = xgb.predict(X_test)
```

***Figure 7.19 Screenshot of Training Vs Test Accuracy of XGBoost***

The XGBoost Classifier model to the training data (X_train and y_train). During this process, the model learns the patterns and relationships present in the training data. XGBoost is an ensemble learning technique that combines numerous weak learners (trees) to make accurate predictions.
The fitting procedure entails an iterative training phase in which the model seeks to minimise a given loss function by updating the weights of the individual decision trees. This optimisation process is repeated until the stated number of boosting rounds (n_estimators) is reached or no substantial improvement in performance is detected.

After fitting the model, it is ready to make predictions on new data using the predict() method. For example, you can use the fitted model to forecast the target variable for the test data.

# 8 TESTING

The testing part of the research on URL-based phishing detection using machine learning algorithms was carefully planned and implemented to assess the performance and effectiveness of the produced models.

The primary purpose of the experiment was to accurately discriminate between phishing and authentic URLs using machine learning models. The test plan was created to evaluate the models performance extensively utilising multiple performance indicators such as accuracy, f1-score, recall, and precision.

The plans are as follows: To ensure thorough evaluation, divide the dataset into training, validation, and testing sets. Testing the models on a different test set of unseen data to judge their generalisation abilities, as well as assessing the models' scalability and efficiency with larger datasets, utilising Grid Search to optimise the model parameters.

End-users were directly involved in the testing phase, and feedback on the final research findings was addressed to ensure practical applicability.

Testing the model in real-time with input URLs. Obtain the URL to be tested. You can either enter a URL directly or use a sample URL from your dataset or another trustworthy source.

Use the trained machine learning model to predict whether the input URL is legitimate or phished. To acquire the predicted class label, pass the encoded characteristics of the input URL to the model.

The testing phase produced promising results, with Random Forest machine learning models displaying good accuracy, f1-score, recall, and precision in detecting phishing URLs. The test strategy was found to be useful in evaluating model performance and finding resilience in various scenarios. The performance of machine learning models using a range of features retrieved from URL-based data was the major focus of testing. The models resistance to various types of URL-based phishing was evaluated.

In a real-world application, a web-based or graphical user interface (GUI) is provided where users can enter a URL and the system will decide whether it is legitimate or phishing based on the trained machine learning model.




***Figure 8.1: Final Testing.***

# 9 PROJECT MANAGEMENT

## 9.1 PROJECT SCHEDULE

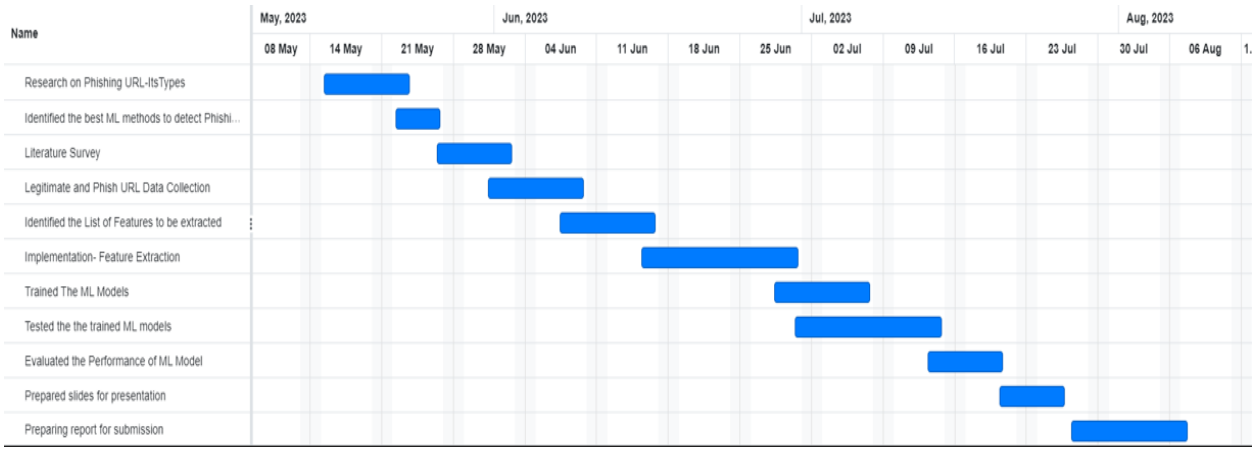



***Figure 9.1 Gantt Chart***

## 9.2 RISK MANAGEMENT

The project comes under a low risk-involved project as the data involved has no security issues or involves no personal data that has the greater possibility of getting misused. The project has data that are taken from public resources such as www.phishtank.org. Only the names of URLs are collected that have nothing to do with personal or institutional risk. The algorithms used are commonly used machine learning algorithms available in open source python libraries and the training resources are low too. The whole project has been done in a free version of Google Colab.

## 9.3 QUALITY MANAGEMENT

The project developed here is a typical back-end project that has not much quality consideration involved other than simplicity of implementation and testing. No front-end or user interface has been designed and developed. Only consideration in the quality management was the easiness and readability of the code developed and the workability of the model and finally the model's reliability in making accurate phishing predictions. All these considerations have been met in the final project.

The project along with the working code has been reviewed frequently to make sure that it's forwarding in the right path. Meeting has been conducted to discuss and reflect on the weekly progresses and the whole project plan has been fine-tuned to perfection till the end.

## 9.4 SOCIAL, LEGAL, ETHICAL AND PROFESSIONAL CONSIDERATIONS

Phishing is a cyberworld-social issue and it has been there ever since the inception of the internet. Developing an accurate solution for detecting, defending, and mitigating phishing will be a social good. The whole project is planned to be open source so that anyone can access the code and model for future projects and improvements without any legal boundaries.

# 10 CRITICAL APPRAISAL

Throughout this project, I have evaluated the work and its effects, both positive and negative, in order to obtain an improved understanding of the subject. This has been a great experience to work closely and hands-on a real world problem solving project. This has been an end to end project beginning right from the research data collection to the final model testing and evaluation. I am sure that the experience gained through out the months of development will help me professionally.

The detailed investigation and implementation of numerous machine learning algorithms, such as KNN, SVM, Decision Trees, Random Forest, Logistic Regression, and XGBoost, is one of the research's strengths. The comparative analysis enabled me to evaluate their performance and choose the best models for the task.

Another advantage is the usage of a big dataset with 70,754 data samples but was unevenly divided between phishing and legitimate URLs. This unbalanced dataset allowed for a thorough assessment of the models' ability to distinguish between the two classes.

However, there were certain difficulties found during the research, the most notable of which was the imbalanced dataset. The equal distribution of phishing and legal samples may have influenced the model's performance, potentially leading to biases. I used oversampling and undersampling strategies to address this problem.

The study also emphasises the significance of feature engineering, as the selected 31 characteristics were critical to the model's success. It is critical for accurate phishing detection to extract relevant and useful aspects from data.

The use of outdated or insufficient datasets can result in biassed models that do not generalise adequately to new and developing phishing threats. Real-world phishing data may be challenging to get for training and testing machine learning models. Because phishing and legitimate websites often have similar qualities, it can be difficult to uncover particular features that consistently distinguish them.

The selection of relevant characteristics is required for the creation of an effective machine learning model. Choosing which features to keep and which to drop, on the other hand, can be a time-consuming and iterative process.

Performance criteria like as accuracy, f1-score, recall, and precision were employed in model evaluation, offering a thorough assessment of the models' strengths and limitations.

During the study process, I learnt important lessons about time management, collaboration, and the importance of in-depth analysis. Furthermore, the need of contemplating future work and modifications to improve the model's capabilities has been emphasised.

# 11 CONCLUSIONS

This study compared multiple machine learning algorithms for identifying URL-based phishing. The goal was to find the most effective model for discriminating between real and phishing URLs. The dataset contained 70,754 samples, with an equal distribution of 35,377 phishing and 35,377 legal samples. Only 10,000 data samples are selected from the original dataset to simplify the training procedure and reduce computing complexity. A total of 31 features were used, with the label "1" representing legal URLs and "-1" representing phishing URLs.

The performance of six distinct machine learning models, namely Random Forest, XGBoost Classifier, Decision Tree, Support Vector Machine (SVM), K-Nearest Neighbours (KNN), and Logistic Regression, was examined using numerous measures such as accuracy, f1 score, recall, and precision. Among all models, Random Forest was found to be the best with an accuracy of 96.8% and XGBoost Classifier earned an accuracy of 96.5%. These two models outperformed the others and exhibited their ability to correctly categorise URLs as phishing or real.

## 11.1 ACHIEVEMENTS

The data collection of the project has been a difficult job in research and decision making as well. Because, there are a number of repositories (many of them are open source) with phishing URLs and there was a lot of confusion about how to choose the right dataset. It took alot of research to decide to collect the dataset myself so that we will get the fresh phishing website URLs and a best performing model after training.

So, the dataset collected, preprocessed, feature extracted and used in this project is a new phishing URL dataset that can be open sourced and can be used for future researches and projects.

Successfully developed a machine learning model capable of detecting phishing URLs with good accuracy, precision, recall, and F1 score. A comprehensive comparison of several machine learning algorithms, including KNN, SVM, Decision Tree, Random Forest, and Logistic Regression, was carried out.

## 11.2 FUTURE WORK

Website phishing is the biggest threat in the cyber security space! Yet, there is not much defense system present that has the characteristics such as high accuracy and light weight. We collected data for both the problems and carried out an extensive preprocessing on it to make it a machine learning trainable dataset.

As a future thought, the model developed may be deployed as a callable open source python API so that anyone can access it anytime. Deep learning approaches, such as deep neural networks or transformer models, can also be used to exploit enormous amounts of data and learn complicated patterns for better detection. Addressing the issue of imbalanced datasets by using approaches such as oversampling, undersampling, or producing synthetic samples to improve the model's performance on minority class recognition.

# 12 STUDENT REFLECTIONS

Throughout this research project, I spent a lot of time researching about URL-based phishing detection and machine learning methods. However, I ran into issues, most notably an imbalanced dataset. Using obsolete or inadequate datasets can lead to biassed models that do not generalise well to new and evolving phishing threats. It might be difficult to obtain real-world phishing information for training and testing machine learning models. Phishing and genuine websites can have comparable characteristics, making it difficult to find specific features that can consistently distinguish them.

Creating an effective machine learning model requires the selection of relevant features. Choosing which features to include and which to eliminate, on the other hand, can be an exhausting and iterative process. To fix this, I used Python's imbalanced-learn module using oversampling and undersampling approaches. The necessity of feature engineering and extensive model evaluations was taught. Another important takeaway was effective time management. I intend to investigate deep learning models in the future and create a more user-friendly API. Overall, this research has been a fantastic learning experience that has fueled my desire to continue growing and contributing in the field of cybersecurity.

# REFERENCES

[1] E. Sandi, “A Survey of URL-based Phishing Detection,” pp. 1–8, 2019.

[2] J. James, Sandhya L. and C. Thomas, "Detection of phishing URLs using machine learning techniques," 2013 International Conference on Control Communication and Computing (ICCC), Thiruvananthapuram, India, 2013, pp. 304-309, doi: 10.1109/ICCC.2013.6731669.

[3] AbuTaha, M., Ababneh, M., Mahmoud, K.W., & Baddar, S.W. (2021). URL Phishing Detection using Machine Learning Techniques based on URLs Lexical Analysis. 2021 12th International Conference on Information and Communication Systems (ICICS), 147-152

[4] M. M. Uddin, K. Arfatul Islam, M. Mamun, V. K. Tiwari and J. Park, "A Comparative Analysis of Machine Learning-Based Website Phishing Detection Using URL Information," 2022 5th International Conference on Pattern Recognition and Artificial Intelligence (PRAI), Chengdu, China, 2022, pp. 220-224, doi: 10.1109/PRAI55851.2022.9904055.

[5] Nandi Tultul, A., Afroz, R., & Hossain, M. A. (2022). Comparison of the efficiency of machine learning algorithms for phishing detection from uniform resource locator. Indonesian Journal of Electrical Engineering and Computer Science, 28(3), 1640. https://doi.org/10.11591/ijeecs.v28.i3.pp1640-1648

[6]. Megha Agarwal, Shruti Jani, Hansika Koli, “Phishing Website Detection Using Machine Learning ”, International Research Journal of Engineering and Technology (IRJET) e-ISSN: 2395-0056 Volume: 10 Issue: 01 | Jan 2023.

[7]. S. Alrefaai, G. Özdemir and A. Mohamed, "Detecting Phishing Websites Using Machine Learning," 2022 International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA), Ankara, Turkey, 2022, pp. 1-6, doi: 10.1109/HORA55278.2022.9799917.

[8]. Y. Xu et al., "A Phishing Website Detection and Recognition Method Based on Naive Bayes," 2022 IEEE 6th Information Technology and Mechatronics Engineering Conference (ITOEC), Chongqing, China, 2022, pp. 1557-1562, doi: 10.1109/ITOEC53115.2022.9734474.

[9]. Ahmed J et.al. “An adaptive approach for internet phishing detection based on log data”. ISSN 2303-4521 Periodicals of Engineering and Natural Sciences Case Study Vol. 9, No. 4, October 2021, pp.622-631.

[10]. N. Q. Do, A. Selamat, O. Krejcar, E. Herrera-Viedma and H. Fujita, "Deep Learning for Phishing Detection: Taxonomy, Current Challenges and Future Directions," in IEEE Access, vol. 10, pp. 36429-36463, 2022, doi: 10.1109/ACCESS.2022.3151903.

[11]. Zainab Alshing, et.al. “A Deep Learning-Based Phishing Detection System Using CNN, LSTM, and LSTM-CNN” Electronics 2023, 12(1), 232; https://doi.org/10.3390/electronics12010232.

[12]. Jibrilla Tanimu. et.al. “Phishing Detection Using Machine LearningAlgorithm”. Conference: 2022 IEEE International Conference on Cyber Security and Resilience (CSR) WorkshopsAt: Virtual Conference.

[13]. Buket Geyik et al.“Detection of Phishing Websites from URLs by using Classification Techniques on WEKA” January 2021 DOI:10.1109/ICICT50816.2021.9358642 Conference: 2021 6th International Conference on Inventive Computation Technologies (ICICT).

[14]. Masurkar, Siddhesh & Dalal, Vipul. (2020). ENHANCED MODEL FOR DETECTION OF PHISHING URL USING MACHINE LEARNING. 158-163. 10.26480/etit.02.2020.158.163.

[15]. Sundara Pandiyan S, Prabha Selvaraj, Vijay Kumar Burugari, Julian Benadit P, Kanmani P, “Phishing attack detection using Machine Learning”, Measurement: Sensors, Volume 24, 2022, 100476, ISSN 2665-9174, https://doi.org/10.1016/j.measen.2022.100476.

[16]. Bhavani, P. Amba and Chalamala, Madhumitha and Likhitha, Pinnam Sree and Sai, Chanda Pranav Sai, Phishing Websites Detection Using Machine Learning (September 2, 2022). Available at SSRN: https://ssrn.com/abstract=4208185 or http://dx.doi.org/10.2139/ssrn.4208185.

[17]. Alnemari, S.; Alshammari, M. Detecting Phishing Domains Using Machine Learning. Appl. Sci. 2023, 13, 4649. https://doi.org/10.3390/app13084649

[18]. Arun Kulkarni, & Leonard L. Brown III (2019). Phishing Websites Detection using Machine Learning. *International Journal of Advanced Computer Science and Applications, 10(7).*

[19]. Catal, C., Giray, G., Tekinerdogan, B. *et al.* Applications of deep learning for phishing detection: a systematic literature review. *Knowl Inf Syst* **64**, 1457–1500 (2022). https://doi.org/10.1007/s10115-022-01672-x

[20]. Adebowale, Moruf & Lwin, Khin & Hossain, Alamgir. (2020). Intelligent Phishing Detection Scheme Algorithms Using Deep Learning. Journal of Enterprise Information Management. ahead-of-print. 10.1108/JEIM-01-2020-0036.

[21]. Yang R, Zheng K, Wu B, Wu C, Wang X. Phishing Website Detection Based on Deep Convolutional Neural Network and Random Forest Ensemble Learning. Sensors (Basel). 2021 Dec 10;21(24):8281. doi: 10.3390/s21248281. PMID: 34960375; PMCID: PMC8709380.

[22]. Ping Yi ,Yuxiang Guan, Futai Zou, Yao Yao, Wei Wang, Ting Zhu. “Web Phishing Detection Using a Deep Learning Framework”.

[23]. Yun Lin, Ruofan Liu, Dinil Mon Divakaran, Jun Yang Ng, Qing Zhou Chan, Yiwen Lu, Yuxuan Si, Fan Zhang, & Jin Song Dong (2021). Phishpedia: A Hybrid Deep Learning Based

Approach to Visually Identify Phishing Webpages. In *30th USENIX Security Symposium (USENIX Security 21)* (pp. 3793–3810). USENIX Association.

[24]. Ariyadasa, Subhash & Fernando, Subha & Fernando, Shantha. (2020). Detecting phishing attacks using a combined model of LSTM and CNN. International Journal of ADVANCED AND APPLIED SCIENCES. 7. 56-67. 10.21833/ijaas.2020.07.007.

[25]. Roy, S.S.; Awad, A.I.; Amare, L.A.; Erkihun, M.T.; Anas, M. Multimodel Phishing URL Detection Using LSTM, Bidirectional LSTM, and GRU Models. Future Internet 2022, 14, 340. https://doi.org/10.3390/fi14110340

# APPENDIX A – PROJECT SPECIFICATION

The deployed code is uploaded in Github and link is given below:

https://github.coventry.ac.uk/samuell/URL-Detection

# APPENDIX B – INTERIM PROGRESS REPORT AND MEETING RECORDS

| Date | Time | Discussed Topic |
|---|---|---|
| 26-05-2023 | 10:30am -11:30am | • Discussed the topic – comparative study of URL based phishing detection using ML and DL methods<br>• Asked perform detailed literature survey in this area |
| 23-06-2023 | 09:30am – 10:30am | • Asked to change the title<br>• To compare various machine learning methods instead of performing both ML and Dl as the time period was short<br>• Include more literature survey papers in the proposal |

## APPENDIX C– PROJECT PRESENTATION

# A COMPARATIVE STUDY OF MACHINE LEARNING METHODS FOR URL-BASED PHISHING DETECTION

7030 CEM CYBER SECURITY INDIVIDUAL PROJECT
SUPERVISED BY: CHRISTO PANCHEV
SUBMITTED BY: LIJI SAMUEL
STUDENT ID: 13163231

## CONTENTS


- INTRODUCTION
- OBJECTIVES
- LITERATURE REVIEW
- ARCHITECTURE
- METHODOLOGY
- DATA COLLECTION
- FEATURE EXTRACTION
- MACHINE LEARNING MODEL BUILDING
- MACHINE LEARNING MODELS
- COMPARISON
- RESULT
- CONCLUSION
- FUTURE SCOPE
- REFERENCE

# INTRODUCTION

- Website Phishing is a cyber security fraud in which the attacker will be faking a legitimate website by its content.
- Also known as URL Phishing
- In 2022, Anti- Phishing Working Group (APWG) observed 1,025,968 phishing attacks.
- IBM's 2022 cost of data breach report found that the average cost of data breach increased from $4.24m in 2021 to $4.35m in 2022.
- Detected using HTML code, URL etc

# OBJECTIVES

- Collect phishing and legitimate URL data using python web scraping tools and extract the URL features from the dataset.
- Clean, preprocess, prepare the dataset to train the ML models.
- Develop and train the model using prepared data.
- Test and evaluate the performance .of the machine learning models on different parameters.

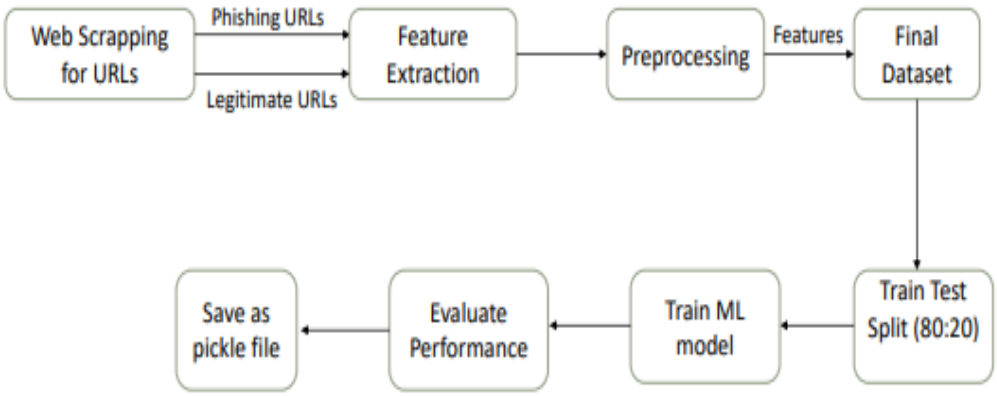
# LITERATURE SURVEY

| Reference | Approach Used | Number of Features | Dataset Used | Accuracy | Pros | Cons |
|---|---|---|---|---|---|---|
| [1] | Random Forest, Gradient Boosting, SVM (Support Vector Machine) | 22 | 1056937 | SVM - 99.896% | • High results in handling the imbalanced dataset | • Difficulty in training |
| [2] | Logistic Regression, K neighbors Classifier, Random Forest, Decision Tree, and XGBoost | 48 | Million samples | XGBoost – 99% | • High performance when more data used as training data | • Million samples - difficulty in training |
| [3] | Decision Tree (DT), Random Forest (RF), XGBoost, Multilayer Perceptrons, K-Nearest Neighbors, Naive Bayes, AdaBoost, and Gradient Boosting | 87 | 11430 | XGBoost - 96.4% | • Faster training and testing<br>• Reduced overfitting | • Small dataset-URLs |

| Reference | Approach Used | Number of Features | Dataset Used | Accuracy | Pros | Cons |
|---|---|---|---|---|---|---|
| [4] | Naïve Bayes algorithm | 22 | 5471 | 74% | • Modified algorithm<br>• Less features extracted | • Old dataset<br>• Increase the URL data training set |
| [5] | SVM, Naïve Bayes, Decision Tree, K-NN, Adaptive Random Forest algorithm (ARFA) | 63 | 96012 | ARFA - 96.91% | • Uses modified algorithm<br>• Low error rate | • Difficulty in training as number of samples is more |
| [6] | ML and Advanced Deep Learning models | | Millions of samples | DNN - 96.38% | • It offers ease of use and implementation | • Time complexity<br>• Limited storage |
| [7] | Long short-term memory (LSTM) , convolutional neural network (CNN) and LSTM-CNN | 80 | 20000 | CNN – 99.2% | • Relevant only to URL characteristics of the same dataset. | • Does not check status of URL website<br>• Low speed |

| Reference | Approach Used | Number of Features | Dataset Used | Accuracy | Pros | Cons |
|---|---|---|---|---|---|---|
| [8] | Decision tree, Support Vector Machine, Naïve base, 2D and 3D Neural Networks, TensorFlow Artificial General Intelligence (AGI) | 30 | 30000 | | • Improved efficiency and efficacy<br>• Feature elimination | • Low recognition speed |
| [9] | Decision Tree (DT), Random Forest (RF), Logistic Regression, Naïve Bayes algorithm | 15 | 126077 | Random Forest - 83% | • Balanced and enhanced dataset can create better solutions | • Longer execution time |
| [10] | Random Forest, Naïve Bayes, SVM, XGBoost, Gradient Boosting | 23 | 13000 | XGBoost - 99.8% | • High accuracy achieved in less time | • Limited set of features |

# ARCHITECTURE



# METHODOLOGY

1. Data collection using web scraping and other sources
2. Feature extraction
3. Data cleaning and preprocessing
4. Develop ML models
5. Train ML models
6. Evaluate the performances using metrics
7. Test and evaluate the best model

# DATA COLLECTION

- 2 types of URLS:
  - **Legitimate URL** - Easy resources such as google index, alexa etc.
  - Legitimate URL source - Cyber Security research of University of New Brunswick (https://www.unb.ca/cic/datasets/url-2016.html ).
  - 35337 legitimate URLs collected from University of New Brunswick
  - **Phishing URL** - Phishing URLs should be live/fresh/latest
  - 78505 phishing URLs collected from Phishtank.com

# FEATURE EXTRACTION

**1) Address Bar Based Features –**

- Domain of URL
- IP Address in URL
- "@" symbol in URL
- Length of URL
- Depth of URL
- Redirection "//" in URL
- "http / https" in Domain name
- Using URL shortening services "TinyURL"
- Prefix or suffix "-" in Domain

# FEATURE EXTRACTION contn…

**2) Domain Based Features –**

- DNS Record
- Website Traffic
- Age of Domain
- End period of Domain
- Page Rank
- Google index
- Status Report

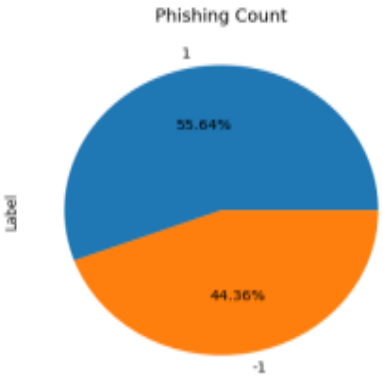
# FEATURE EXTRACTION contn…

**3) HTML & Javascript Related Features -**

- IFrame Redirection
- Status Bar Customization
- Disabling Right Click
- Website Forwarding
- Link in script Tag
- Server from handler
- Using pop-up windows
- Link pointing to page
- HTTPS response
- Non –standard port
- Info mail
- Favicon
- Request URL
- Abnormal URL
- Anchor URL

# MACHINE LEARNING MODEL BUILDING

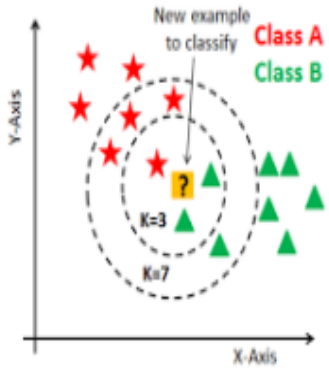
- Took 10000 data to make training easy.
- No outliers and missing values.
- Split the data into training set and test set in the ratio 80:20.

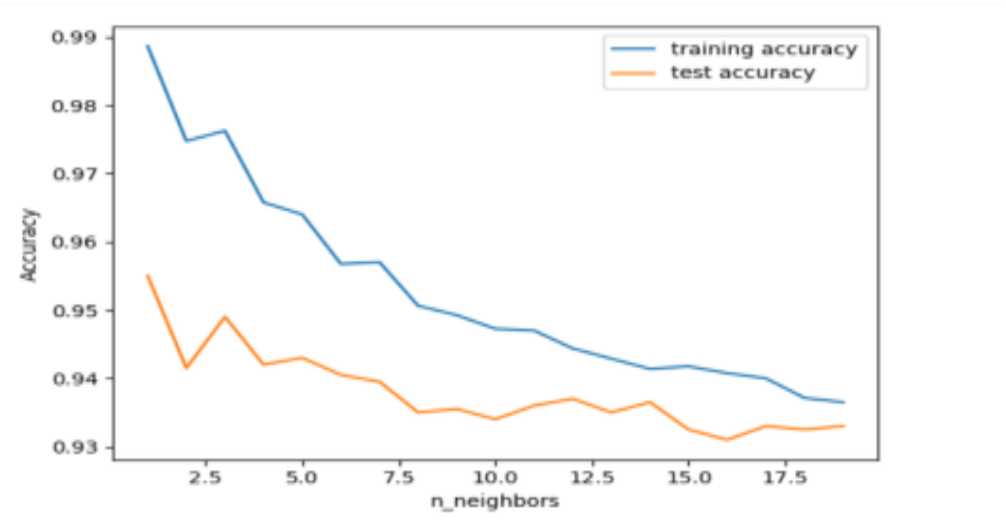


# MACHINE LEARNING MODELS

## 1) MODEL 1 : KNN (K- NEAREST NEIGHBOR) ALGORITHM

- A supervised ML algorithm for classification.
- It makes predictions based on the similarity of data points to other neighboring data points.



```
K-Nearest Neighbors : Accuracy on training Data: 0.989
K-Nearest Neighbors : Accuracy on test Data: 0.955

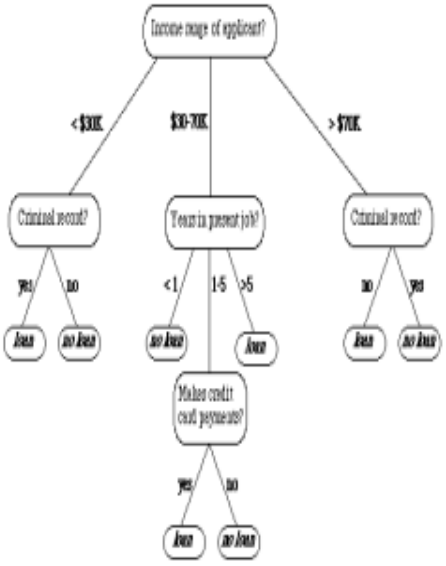
K-Nearest Neighbors : f1_score on training Data: 0.990
K-Nearest Neighbors : f1_score on test Data: 0.960

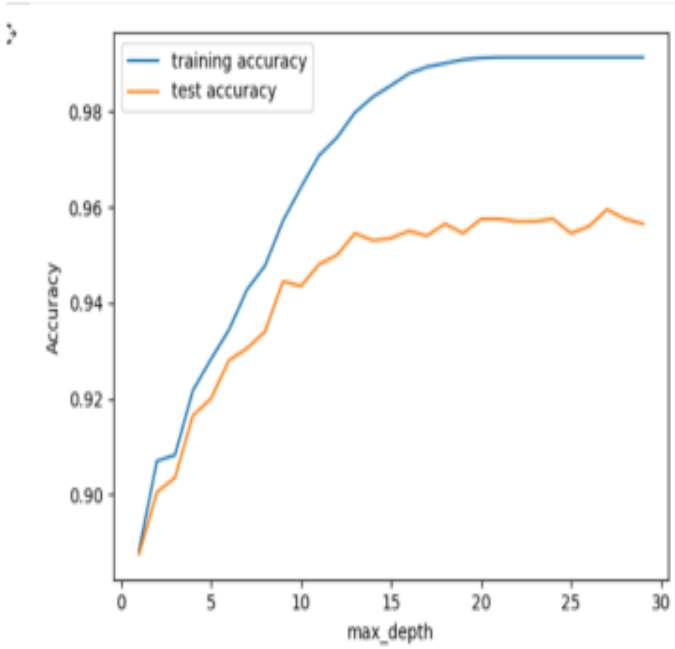
K-Nearest Neighbors : Recall on training Data: 0.990
Logistic Regression : Recall on test Data: 0.962

K-Nearest Neighbors : precision on training Data: 0.990
K-Nearest Neighbors : precision on test Data: 0.957
```

## MODEL 2: DECISION TREE ALGORITHM

- A condition-based algorithm.
- Checks the condition is met or not. Then decide the output.






```
Decision Tree : Accuracy on training Data: 0.991
Decision Tree : Accuracy on test Data: 0.958

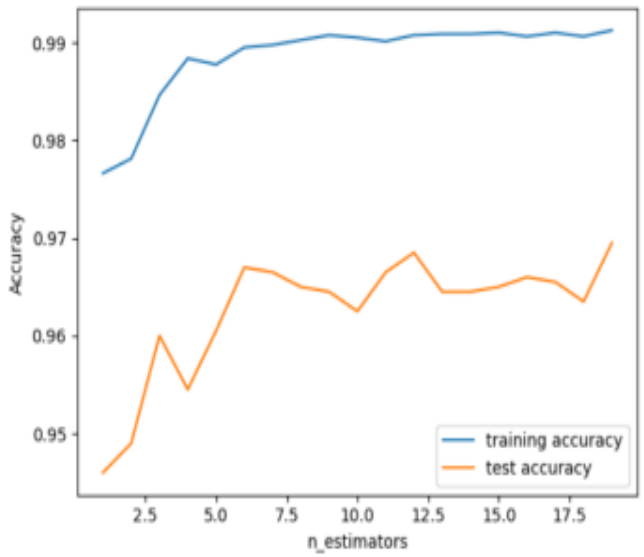
Decision Tree : f1_score on training Data: 0.992
Decision Tree : f1_score on test Data: 0.962

Decision Tree : Recall on training Data: 0.992
Decision Tree : Recall on test Data: 0.962

Decision Tree : precision on training Data: 0.993
Decision Tree : precision on test Data: 0.962
```

## MODEL 3: RANDOM FOREST

- A supervised ML algorithm for classification.
- An ensemble algorithm in which a number of models make prediction at the same time.
- Random forest = n number of decision trees making predictions
- It use decision tree algorithms as the base algorithm.
- We define n = number of decision trees to make predictions and make final prediction based on voting (the class predicted by more trees).



```
Random Forest : Accuracy on training Data: 0.991
Random Forest : Accuracy on test Data: 0.969

Random Forest : f1_score on training Data: 0.992
Random Forest : f1_score on test Data: 0.972

Random Forest : Recall on training Data: 0.994
Random Forest : Recall on test Data: 0.971

Random Forest : precision on training Data: 0.990
Random Forest : precision on test Data: 0.962
```

## MODEL 4: SVM

- A Classic ML model that optimizes a hyperplane (support vector) that separates two classes of data.
- The algorithm finds an optimal hyperplane for the training data.
- It require very less training data to achieve good accuracy.



```
Support Vector Machine : Accuracy on training Data: 0.969
Support Vector Machine : Accuracy on test Data: 0.956

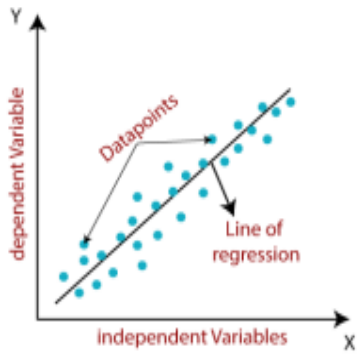
Support Vector Machine : f1_score on training Data: 0.973
Support Vector Machine : f1_score on test Data: 0.961

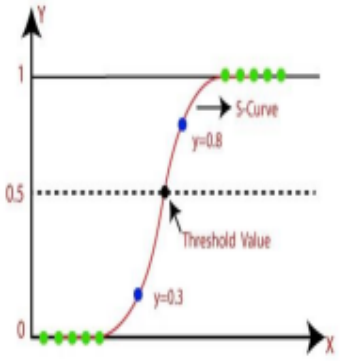
Support Vector Machine : Recall on training Data: 0.979
Support Vector Machine : Recall on test Data: 0.967

Support Vector Machine : precision on training Data: 0.966
Support Vector Machine : precision on test Data: 0.955
```

## MODEL 5: LOGISTIC REGRESSION

- Linear Regression - Find a line that passes closer to all possible data points.
- Logistic Regression - Linear Regression plus a probability threshold



```
Logistic Regression : Accuracy on training Data: 0.927
Logistic Regression : Accuracy on test Data: 0.924

Logistic Regression : f1_score on training Data: 0.935
Logistic Regression : f1_score on test Data: 0.932

Logistic Regression : Recall on training Data: 0.944
Logistic Regression : Recall on test Data: 0.940

Logistic Regression : precision on training Data: 0.927
Logistic Regression : precision on test Data: 0.925
```

## MODEL 6: XGBOOST



```
XGBoost Classifier : Accuracy on training Data: 0.988
XGBoost Classifier : Accuracy on test Data: 0.965

XGBoost Classifier : f1_score on training Data: 0.989
XGBoost Classifier : f1_score on test Data: 0.969

XGBoost Classifier : Recall on training Data: 0.992
XGBoost Classifier : Recall on test Data: 0.992

XGBoost Classifier : precision on training Data: 0.986
XGBoost Classifier : precision on test Data: 0.986
```

# COMPARISON

| | ML Model | Accuracy | f1_score | Recall | Precision |
|---|---|---|---|---|---|
| 0 | Random Forest | 0.968 | 0.971 | 0.993 | 0.990 |
| 1 | XGBoost Classifier | 0.965 | 0.969 | 0.992 | 0.986 |
| 2 | Decision Tree | 0.957 | 0.961 | 0.992 | 0.993 |
| 3 | Support Vector Machine | 0.956 | 0.961 | 0.979 | 0.966 |
| 4 | K-Nearest Neighbors | 0.955 | 0.960 | 0.990 | 0.990 |
| 5 | Logistic Regression | 0.924 | 0.932 | 0.944 | 0.927 |

# RESULT




# CONCLUSION

- The extracted features play important role in the detection accuracy.
- 6 ML models were trained in real time.
- Random forest has the best testing accuracy followed by XGBoost and Decision Tree.
- Logistic Regression has the lowest performance.

# FUTURE SCOPE

- The Accuracy of the models can be increased by extracting more features.
- Other Deep Learning can be used for training and check if they outperform the ML model.
- The developed model can be hosted as a callable API so that anyone can access it anytime for phishing detection or can be used at the backend of a software as well.
- The trained model can be deployed as a browser extension (such as Chrome extension) so that phishing can be detected in real time.

# REFERENCE

[1] AbuTaha, M., Ababneh, M., Mahmoud, K.W., & Baddar, S.W. (2021). URL Phishing Detection using Machine Learning Techniques based on URLs Lexical Analysis. *2021 12th International Conference on Information and Communication Systems (ICICS)*, 147-152.

[2] Megha Agarwal, Shruti Jani, Hansika Koli, "Phishing Website Detection Using Machine Learning ", International Research Journal of Engineering and Technology (JETIR) e-ISSN: 2395-0056 Volume: 10 Issue: 01 | Jan 2023

[3] S. Alrefaai, G. Özdemir and A. Mohamed, "Detecting Phishing Websites Using Machine Learning," 2022 International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA), Ankara, Turkey, 2022, pp. 1-6, doi: 10.1109/HORA55278.2022.9799917.

[4] Y. Xu *et al*., "A Phishing Website Detection and Recognition Method Based on Naive Bayes," *2022 IEEE 6th Information Technology and Mechatronics Engineering Conference (ITOEC)*, Chongqing, China, 2022, pp. 1557-1562, doi: 10.1109/ITOEC53115.2022.9734474.

[10] Masurkar, Siddhesh, Dalal, Vipul. (2020). ENHANCED MODEL FOR DETECTION OF PHISHING URL USING MACHINE LEARNING. 158-163. 10.26480/etit.02.2020.158.163.

# APPENDIX D – CERTIFICATE OF ETHICS APPROVAL






## Certificate of Ethical Approval

| | |
|---|---|
| Applicant: | Liji Samuel |
| Project Title: | A Comparative Study of Machine Learning and Deep Learning Methods for URL-based Phishing Detection |

This is to certify that the above named applicant has completed the Coventry University Ethical Approval process and their project has been confirmed and approved as Low Risk

| | |
|---|---|
| Date of approval: | 20 Jun 2023 |
| Project Reference Number: | P158822 |